





ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO
Departamento de Engenharia de Computação e Sistemas Digitais

PCS3635 – LABORATÓRIO DE PROJETO DE SISTEMAS DIGITAIS I

PROJETO Aventura Financeira – SEMANA 1

Relato da Bancada BA6 – Turma 6 – Prof. Reginaldo

Data de Emissão: 28 de Outubro de 2026.

Nome: Gabriel Lopes Prodossimo	**Número USP: 15453560**
Nome: Luís Felipe Melegari Pinton	**Número USP: 14592219**
Nome: Sophia Soares Mariano	**Número USP: 15511213**

Sumário

1 INTRODUÇÃO 5

2 ESPECIFICAÇÃO TÉCNICA DO PROJETO 6

2.1 Contexto de Aplicação do Projeto 6

2.2 Objetivos do Projeto 6

2.3 Descrição de Recursos e Custos 6

2.4 Definição da Arquitetura do Sistema 7

2.4.1 Descrição da Arquitetura Comportamental 7

2.4.2 *Storytelling* da Interação entre Usuários e Sistema 7

2.4.3 Descrição da Arquitetura Estrutural 7

2.5 Especificação de Requisitos 8

2.6 Cronograma do Projeto 9

3 SEMANA 1 DE DESENVOLVIMENTO DO PROJETO 10

3.1 Descrição Funcional do Projeto para a Semana 10

3.2 Detalhamento do Projeto Lógico do Hardware Programável 10

3.2.1 Projeto do Fluxo de Dados 10

3.2.2 Projeto da Unidade de Controle 10

3.2.3 Projeto do Sistema Digital Programável 11

3.3 Plano de Testes do Hardware Programável e Simulações 11

3.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 11

3.4 Detalhamento do Projeto de Software 12

3.4.1 Descrição da Arquitetura de Software 12

3.4.2 Detalhamento do Projeto de Software 12

3.5 Plano de Testes do Software 12

3.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 12

3.6 Integração entre Hardware Programável e Software 13

3.7 Prototipação do Projeto 13

3.8 Implantação do Projeto 14

3.8.1 Pinagem da Placa FPGA 14

3.8.2 Estratégia de Montagem 14

3.8.3 Estratégia de Depuração 14

3.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário> 15

3.9 Conclusão da Semana 16

4 SEMANA 2 DE DESENVOLVIMENTO DO PROJETO 17

4.1 Descrição Funcional do Projeto para a Semana 17

4.2 Detalhamento do Projeto Lógico do Hardware Programável 17

4.2.1 Projeto do Fluxo de Dados 17

4.2.2 Projeto da Unidade de Controle 17

4.2.3 Projeto do Sistema Digital Programável 18

4.3 Plano de Testes do Hardware Programável e Simulações 18

4.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 18

4.4 Detalhamento do Projeto de Software 19

4.4.1 Descrição da Arquitetura de Software 19
4.4.2 Detalhamento do Projeto de Software 19
4.5 Plano de Testes do Software 19
4.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 19
4.6 Integração entre Hardware Programável e Software 20
4.7 Prototipação do Projeto 20
4.8 Implantação do Projeto 21
4.8.1 Pinagem da Placa FPGA 21
4.8.2 Estratégia de Montagem 21
4.8.3 Estratégia de Depuração 21
4.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário> 22
4.9 Conclusão da Semana 23
5 SEMANA 3 DE DESENVOLVIMENTO DO PROJETO 24
5.1 Descrição Funcional do Projeto para a Semana 24
5.2 Detalhamento do Projeto Lógico do Hardware Programável 24
5.2.1 Projeto do Fluxo de Dados 24
5.2.2 Projeto da Unidade de Controle 24
5.2.3 Projeto do Sistema Digital Programável 25
5.3 Plano de Testes do Hardware Programável e Simulações 25
5.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 25
5.4 Detalhamento do Projeto de Software 26
5.4.1 Descrição da Arquitetura de Software 26
5.4.2 Detalhamento do Projeto de Software 26
5.5 Plano de Testes do Software 26
5.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 26
5.6 Integração entre Hardware Programável e Software 27
5.7 Prototipação do Projeto 27
5.8 Implantação do Projeto 28
5.8.1 Pinagem da Placa FPGA 28
5.8.2 Estratégia de Montagem 28
5.8.3 Estratégia de Depuração 28
5.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário> 29
5.9 Conclusão da Semana 30
6 SEMANA 4 DE DESENVOLVIMENTO DO PROJETO 31
6.1 Descrição Funcional do Projeto para a Semana 31
6.2 Detalhamento do Projeto Lógico do Hardware Programável 31
6.2.1 Projeto do Fluxo de Dados 31
6.2.2 Projeto da Unidade de Controle 31
6.2.3 Projeto do Sistema Digital Programável 32
6.3 Plano de Testes do Hardware Programável e Simulações 32
6.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 32
6.4 Detalhamento do Projeto de Software 33
6.4.1 Descrição da Arquitetura de Software 33
6.4.2 Detalhamento do Projeto de Software 33

6.5 Plano de Testes do Software **33**
6.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário> 33
6.6 Integração entre Hardware Programável e Software **34**
6.7 Prototipação do Projeto **34**
6.8 Implantação do Projeto **35**
6.8.1 Pinagem da Placa FPGA 35
6.8.2 Estratégia de Montagem 35
6.8.3 Estratégia de Depuração 35
6.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário> 36
6.9 Conclusão da Semana **37**
7 CONCLUSÕES DO PROJETO **38**
8 REFERÊNCIAS BIBLIOGRÁFICAS **39**

1. Introdução



O projeto “Aventura Financeira Evolution” consiste em um jogo de tabuleiro digital interativo que simula uma situação financeira real, onde o jogador pode trabalhar, estudar, investir e comprar coisas dentro do jogo. O jogo permitirá a aprendizagem da gestão de recursos financeiros, uma situação real de vida que vai ser inserida no repertório do jogador de forma natural, com fácil utilização. Assim, busca-se proporcionar ao jogador um conhecimento básico de gestão comportamental de educação financeira, atuando como introdução ao tema.

O tema foi escolhido para suportar pessoas com Transtorno do Espectro Autista tendo em vista a dificuldade de abstrair e compreender o valor real do dinheiro que

geralmente ocorre dentro do público alvo. Além disso, a aversão ao risco é outro tópico relevante para autistas e, por isso, a criação de um simulador no qual o jogador pode fazer decisões sem impacto real é uma solução para o ensino de educação financeira.

2. Definição do Problema, do Escopo e do Público-Alvo

- **Problema:** Pessoas no Transtorno do Espectro Autista (TEA) enfrentam frequentemente desafios relacionados com a literalidade e a dificuldade de abstração, o que as pode levar a cair em armadilhas financeiras no dia a dia. Esta dificuldade no manuseamento do dinheiro gera falta de segurança e uma dependência contínua dos familiares.

- **Público-Alvo:** Jovens e adultos da comunidade TEA que procuram desenvolver autonomia financeira e aprender a gerir os seus gastos.

- **Escopo:** O projeto consiste no desenvolvimento de um jogo educativo sobre educação financeira, processado numa placa FPGA DE0-CV. O sistema simulará situações reais de consumo, trabalho e investimento num ambiente livre de estresse, utilizando comandos literais e aprendizagem rápida. O hardware integrará um *dashboard* no computador para exibir o saldo/métricas do jogador, um sensor para simular a ação de "pagamento" e um atuador para fornecer *feedback* físico (como a libertação de um recibo ou a abertura de uma gaveta de dinheiro).

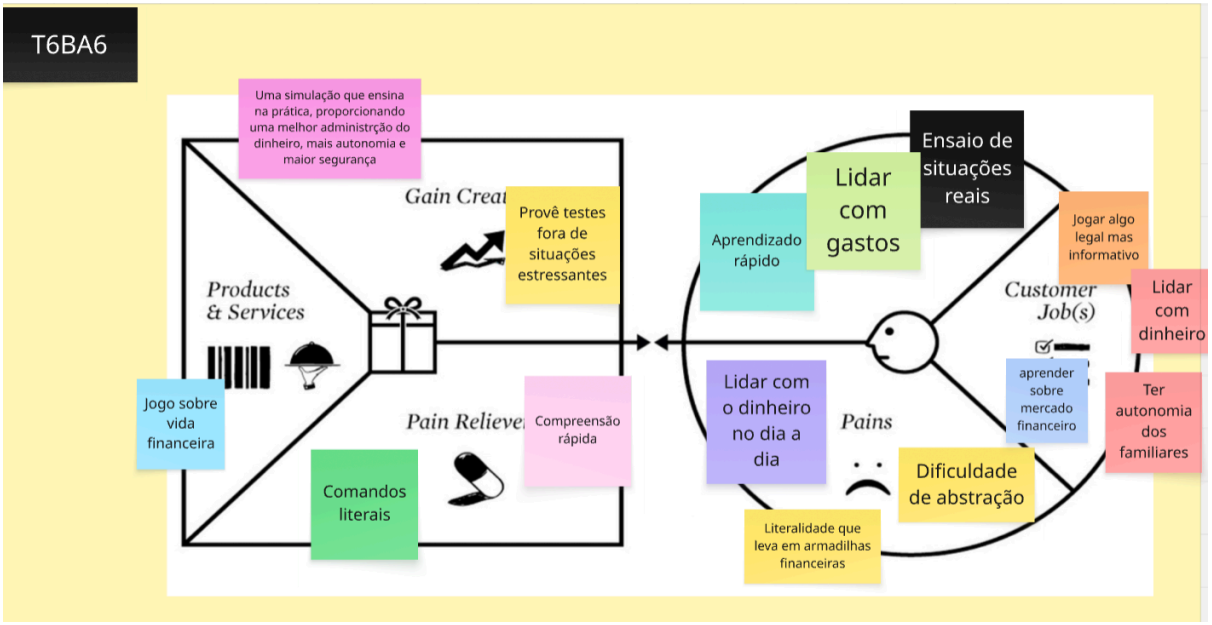


Imagem 1: Especificação dos requisitos da AEX2

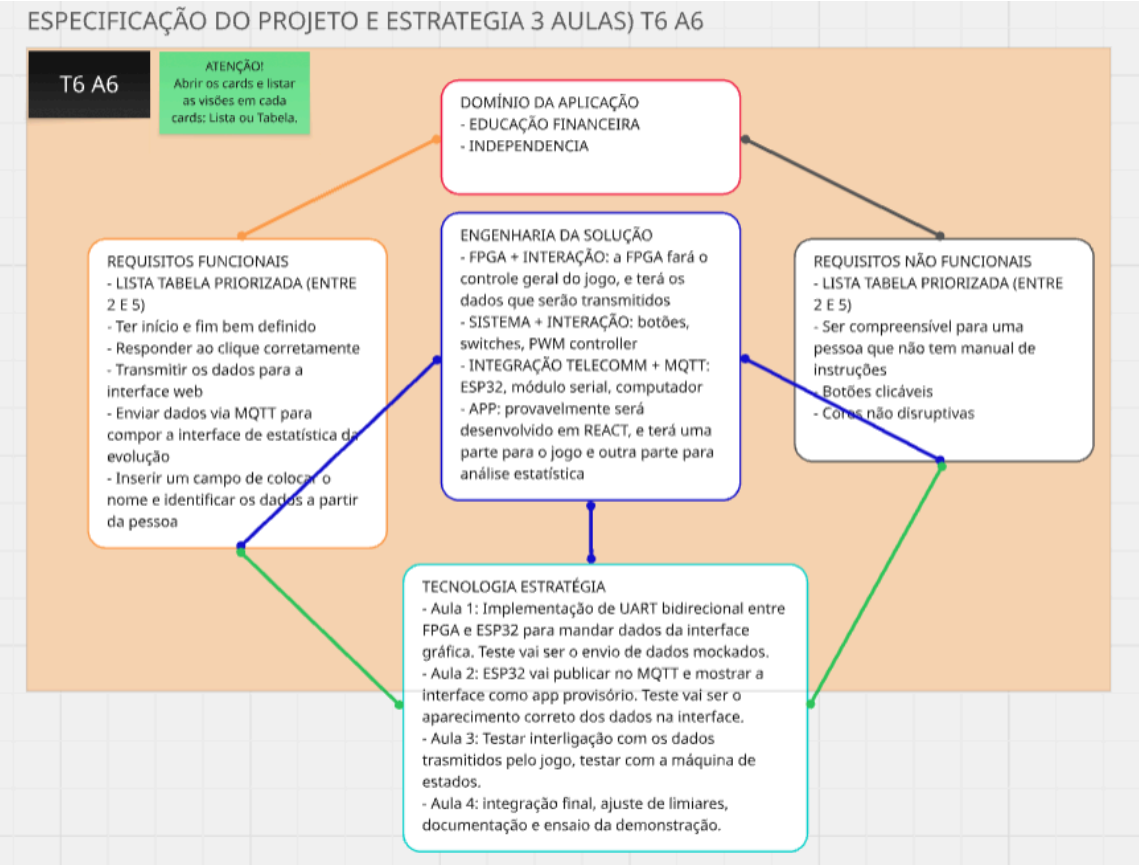


Imagem 2: Especificação dos requisitos da AEX4

2.1. Definição do Orçamento e Recursos Necessários

- **Recursos dados pelo LABDIG:** O projeto utilizará os equipamentos emprestados pelo LabDig (Laboratório de Sistemas Digitais), que incluem: a placa FPGA DE0-CV, uma placa adicional para montagem externa (protoboard), cabos flat, e um servomotor (SG90).

Item	Quantidade	Custo Unitário	Custo Total	Referência
FPGA	1	-	-	-
Caixa de madeira grande	1	R$ 52,99	R$ 52,99	caixa de madeira
Cartão RFID	1	R$ 19,30	R$ 19,30	cartão rfid
Tinta	2	R$ 13,90	R$ 27,80	tinta
Papel Couchê	1	R$ 29,29	R$ 29,29	papel

Mini brinquedos lembrancinha	1	R$ 30,60	R$ 30,60	mini brinquedos
Bombom serenata	1		R$ 0,00	bombom
Alfinete	1	R$ 15,30	R$ 15,30	alfinete
Custo Total do Projeto:			R$ 175,28	

A quantidade de horas esperadas de trabalho semanais por pessoa é de 5h, devido a complexidade do projeto, aliada às montagens físicas e decorativas que devem ser feitas para que a apresentação do projeto fique agradável.

3. Arquitetura Preliminar do Projeto

3.1. Arquitetura Estrutural (Diagrama de Blocos)

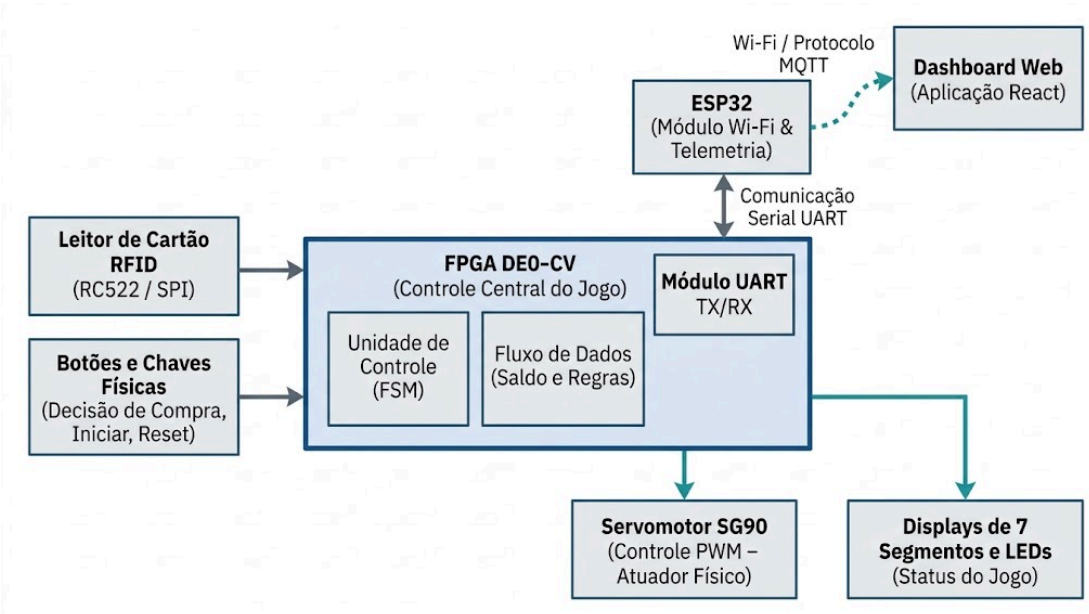


Imagem 1: Diagrama da Arquitetura Estrutural

3.2. Arquitetura Comportamental (Fluxograma)

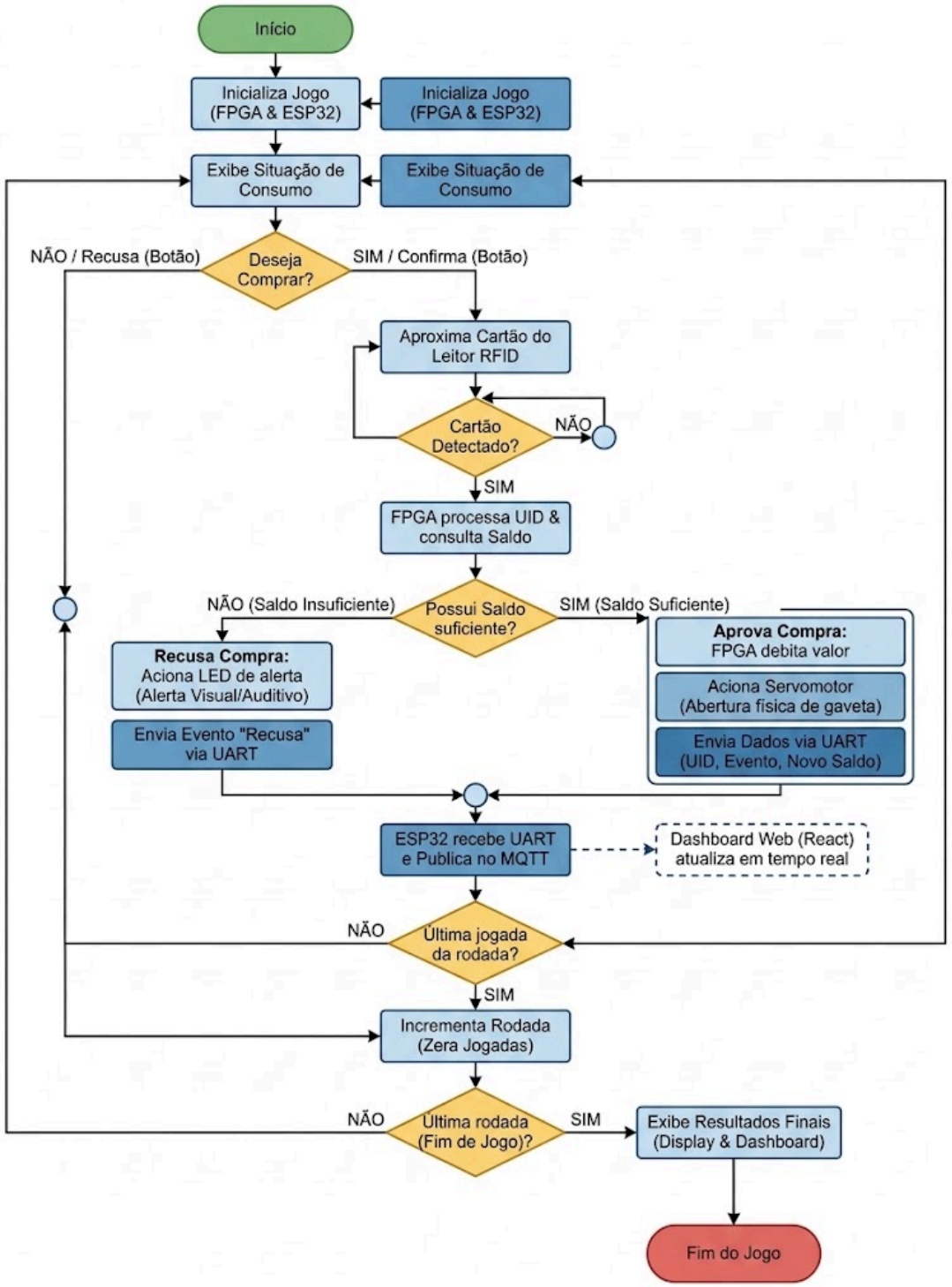


Imagem 2: Diagrama da Arquitetura Comportamental

3.3. Interações externas (Storyboard)

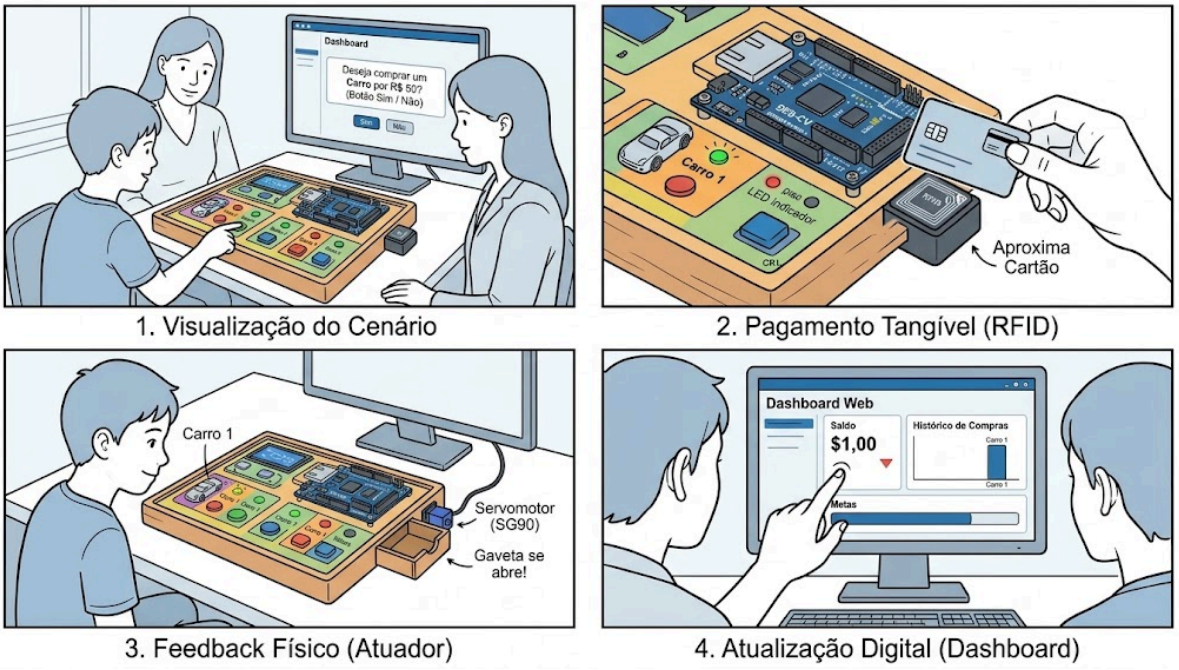


Imagem 3: Storyboard de Interações Externas

4. Especificação de Requisitos

4.1. Requisito 1: Registo e Gestão do Saldo

Código: SALDO_INICIAL	☑ Funcional □ Não Funcional
Requisito: Registro do saldo do jogador.	
Descrição: O sistema deve iniciar a simulação financeira com um valor de saldo pré-definido, que será reduzido de acordo com as escolhas de gastos do utilizador durante o jogo.	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** Este requisito é fundamental para o core do jogo, pois permite ao jogador praticar a administração do dinheiro. Método de verificação: simulação funcional na FPGA para verificar a dedução correta dos valores do saldo.	
Requisitos associados: DISPLAY_DASHBOARD.	

4.2. Requisito 2: Deteção de Pagamento via RFID

Código: DETECCAO_PAGAMENTO	☑ Funcional □ Não Funcional
Requisito: Deteção da ação de pagamento via sensor.	
Descrição: O sistema deve reconhecer a intenção de pagamento quando um objeto (simulando um cartão) for aproximado a 5 cm ou menos do sensor RFID.	

Prioridade:	☑ Alta	□ Média	□ Baixa
Estabilidade:	□ Alta	☑ Média	□ Baixa
***Rationale*:** Cumpre a restrição obrigatória de uso de sensores no projeto e traduz a abstração de um pagamento para uma ação física e literal.			
Requisitos associados: SALDO_INICIAL.			

4.3. Requisito 3: Acionamento Mecânico pós-compra

Código: ACIONA_MOTOR	☑ Funcional	□ Não Funcional
Requisito: Acionamento do atuador pós-compra.		
Descrição: O sistema deve acionar o servomotor (SG90) sempre que o sensor detectar o pagamento e o saldo for suficiente.		
Prioridade: ☑ Alta	□ Média	□ Baixa
Estabilidade: ☑ Alta	□ Média	□ Baixa
***Rationale*:** Cumpre a restrição obrigatória de incorporação de, pelo menos, um atuador no projeto. Fornece um feedback tátil e mecânico concreto, auxiliando a compreensão da relação de causa e efeito para o utilizador TEA, minimizando a necessidade de abstração.		
Requisitos associados: DETECAO_PAGAMENTO, SALDO_INICIAL.		

4.4. Requisito 4: Exibição no Dashboard Web

Código: DISPLAY_DASHBOARD	☑ Funcional	□ Não Funcional
Requisito: Exibição e atualização do Dashboard financeiro.		
Descrição: O sistema deve calcular o novo saldo após cada decisão de consumo e exibir o valor atualizado em tempo real no Dashboard (interface visual).		
Prioridade: ☑ Alta	□ Média	□ Baixa
Estabilidade: ☑ Alta	□ Média	□ Baixa
***Rationale*:** Cumpre a restrição obrigatória de incorporar o cálculo e a exibição de um dashboard com dados do projeto. Permite a visualização literal do dinheiro disponível, resolvendo a dor da "literalidade que leva a armadilhas financeiras".		
Requisitos associados: SALDO_INICIAL, SELECAO_COMPRA.		

4.5. Requisito 5: Interação de Decisão de Compra

Código: SELECAO_COMPRA	☑ Funcional □ Não Funcional
Requisito: Interação de recusa ou aceitação de compra.	
Descrição: O sistema deve permitir ao jogador aceitar ou recusar a simulação de compra do item da rodada através da interação direta com os botões físicos da placa DE0-CV.	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☑ Alta □ Média □ Baixa	
***Rationale*:** Garante a interatividade do jogo processada na lógica central da placa FPGA DE0-CV, permitindo o ensaio de situações reais e a tomada de decisão financeira autónoma, gerando um ambiente de teste livre de stress.	
Requisitos associados: SALDO_INICIAL.	

4.6. Requisito 6: Latência de Resposta

Código: FEEDBACK_IMEDIATO	□ Funcional ☑ Não Funcional
Requisito: Tempo de resposta do sistema após pagamento.	
Descrição: O sistema deve fornecer o feedback da compra (atualização numérica do Dashboard e movimento do motor) num tempo máximo de 1 segundo após a deteção do cartão pelo sensor RFID.	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☐ Alta ☑ Média □ Baixa	
***Rationale*:** Para o público-alvo TEA, atrasos na resposta do sistema podem causar quebra de expectativa, confusão ou ansiedade. Uma resposta de baixa latência reforça a segurança e a aprendizagem rápida da dinâmica financeira.	
Requisitos associados: ACIONA_MOTOR, DISPLAY_DASHBOARD, DETECCAO_PAGAMENTO.	

4.7. Requisito 7: Transmissão Serial UART (FPGA → ESP32)

Código: COM_UART_TX	☑ Funcional □ Não Funcional
Requisito: Envio serial dos eventos de jogo a partir da FPGA.	
Descrição: A FPGA deve integrar um controlador UART transmissor configurado a 115200 bps (8-N-1) para enviar pacotes de telemetria contendo: identificador do evento (compra aprovada, recusa, fim de rodada), identificador do item e valor resultante do saldo para a placa ESP32.	

Prioridade:	☑ Alta	□ Média	□ Baixa
Estabilidade:	☐ Alta	☑ Média	□ Baixa
***Rationale*:** Estabelece o desacoplamento entre o processamento crítico em hardware digital e a camada de rede IoT.			
Requisitos associados: SALDO_INICIAL, TELEMETRIA_MQTT.			

4.8. Requisito 8: Publicação de Dados via MQTT

Código: TELEMETRIA_MQTT	☑ Funcional	□ Não Funcional
Requisito: Publicação em nuvem/rede local dos eventos em broker MQTT.		
Descrição: A ESP32 deve interpretar os dados recebidos pela porta serial da FPGA, formatá-los em formato JSON e publicá-los no tópico aventura_financeira/telemetria de um broker MQTT (ex.: Mosquitto ou HiveMQ) através de conexão Wi-Fi.		
Prioridade:	☑ Alta	□ Média □ Baixa
Estabilidade:	☐ Alta	☑ Média □ Baixa
***Rationale*:** Cumpre o requisito de telecomunicações do projeto e permite o desacoplamento do cliente web de monitorização		
Requisitos associados: COM_UART_TX, DISPLAY_DASHBOARD		

4.9. Requisito 9: Sinalização de Falha/Saldo Insuficiente

Código: ALERTA_RECUSA	☑ Funcional	□ Não Funcional
Requisito: Sinalização clara de recusa de transação por saldo insuficiente.		
Descrição: Caso o jogador tente comprar um item sem ter saldo disponível, a FPGA não deve acionar o motor; deve acender um LED indicador vermelho dedicado e enviar o evento de "compra recusada" via UART para registo no Dashboard.		
Prioridade:	☑ Alta	□ Média □ Baixa
Estabilidade:	☐ Alta	☑ Média □ Baixa
***Rationale*:** Ensina a restrição orçamentária de maneira explícita e consistente, sem penalidades severas, permitindo a tentativa orientada.		
Requisitos associados: SALDO_INICIAL, COM_UART_TX		

4.10. Requisito 10: Persistência e Exportação de Sessão

Código: LOG_SESSAO	☑ Funcional □ Não Funcional
Requisito: Registo dos dados da sessão para acompanhamento pedagógico.	
Descrição: A aplicação de dashboard deve registrar todas as jogadas e transações de uma sessão com identificação de utilizador, permitindo a visualização gráfica final da evolução financeira da partida.	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☐ Alta ☑ Média □ Baixa	
***Rationale*:** Permite que tutores, educadores ou familiares avaliem o progresso cognitivo do jogador na gestão de recursos.	
Requisitos associados: DISPLAY_DASHBOARD, TELEMETRIA_MQTT	

4.11. Requisito 11: Acessibilidade Sensorial para Pessoas com TEA

Código: CONFORTO_SENSORIAL	□ Funcional ☑ Não Funcional
Requisito: Adequação de estímulos visuais e táteis às diretrizes de suporte a TEA.	
Descrição: O dashboard e o tabuleiro físico devem empregar cores suaves, sem padrões estroboscópicos ou transições bruscas de alta intensidade lumínica. Atuadores e eventuais sinais sonoros devem operar em faixas moderadas de frequência e volume para evitar sobrecarga sensorial.	
Prioridade: ☑ Alta □ Média □ Baixa	
Estabilidade: ☐ Alta ☑ Média □ Baixa	
***Rationale*:** Garantir que o ambiente de aprendizagem permaneça acolhedor e livre de gatilhos de hiper-reatividade sensorial inerentes ao público-alvo.	
Requisitos associados: DISPLAY_DASHBOARD, ALERTA_RECUSA	

4.12. Requisito 12: Usabilidade e Clareza Semiótica da Interface

Código: USABILIDADE_LITERAL	□ Funcional ☑ Não Funcional
Requisito: Legibilidade semântica e comunicação direta sem ambiguidades.	

Descrição: Todas as instruções, botões e telas do dashboard devem utilizar linguagem literal, direta e acompanhada de ícones autoexplicativos (ex.: símbolo de cartão para leitura RFID, cores verde/vermelho convencionadas para sim/não), dispensando interpretação conotativa.
Prioridade: ☑ Alta □ Média □ Baixa
Estabilidade: ☐ Alta ☑ Média □ Baixa
***Rationale*:** Reduz o atrito de interpretação e a ambiguidade para utilizadores no espectro autista com perfil de pensamento literal.
Requisitos associados: SELECAO_COMPRA, DISPLAY_DASHBOARD

5. Cronograma

Código	Nome Curto	Tipo	Prioridade	Semana Prevista
SALDO_INICIAL	Registo e Cálculo do Saldo	RF	Alta	Semana 1
SELECAO_COMPRA	Decisão de Compra (Botões)	RF	Alta	Semana 1
COM_UART_TX	Driver Serial UART (FPGA)	RF	Alta	Semana 1 / 2
DETECCAO_PAGAMENTO	Leitura de Cartão RFID	RF	Alta	Semana 2
ACIONA_MOTOR	Acionamento de Gaveta (SG90)	RF	Alta	Semana 2
ALERTA_RECUSA	Tratamento de Falha / Sem Saldo	RF	Alta	Semana 2
TELEMETRIA_MQTT	Publicação ESP32->MQTT	RF	Alta	Semana 2 / 3
DISPLAY_DASHBOARD	Exibição Web em React	RF	Alta	Semana 3
LOG_SESSAO	Histórico e Métricas de Sessão	RF	Média	Semana 3 / 4

COMPRAS_MATERIAIS	Compras	RNF	Alta	Semana 1
FEEDBACK_IMEDIATO	Latência Total	RNF	Alta	Semana 4
CONFORTO_SENSORIAL	Acessibilidade Visual e Tátil	RNF	Alta	Semana 3 / 4
USABILIDADE_LITERAL	Interface e Semiótica Direta	RNF	Alta	Semana 4

1 SEMANA 1 DE DESENVOLVIMENTO DO PROJETO

Fazer o projeto antigo funcionar, revisar o Verilog e a lógica do projeto, entender como funciona o sensor de aproximação.

1.1 Descrição Funcional do Projeto para a Semana

Projeto Aventura Financeira completamente operacional considerando o projeto da disciplina. Nessa primeira semana vamos garantir que esteja tudo funcionando e pensar em como integrar o aproximação para o pagamento e também mostrar os dados através da tela e não do display. Assim, utilizar-se-á a transmissão serial no projeto, objetivo desejado.



Imagem 4: Protótipo e Dashboard em uso

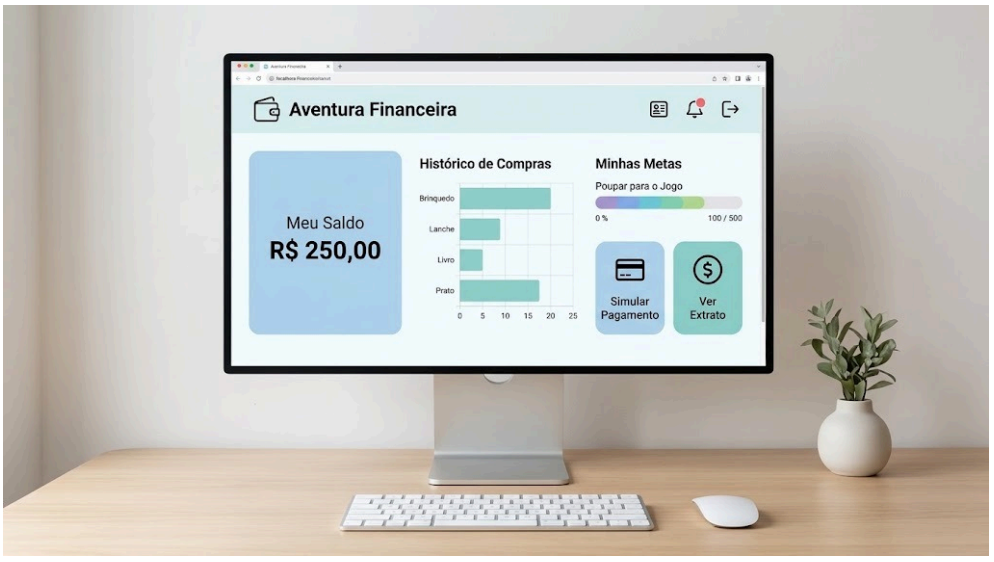


Imagem 5: Interface do Dashboard Web

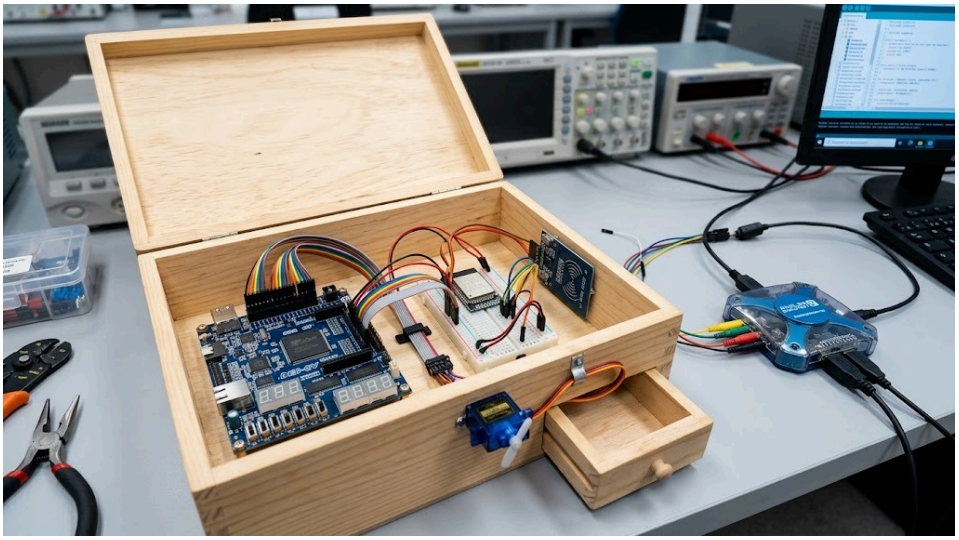


Imagem 6: Montagem do Hardware na caixa

1.2 Detalhamento do Projeto Lógico do Hardware Programável

Esta seção do documento tem a finalidade de descrever detalhes do projeto lógico de hardware programável desenvolvido na Semana 1.

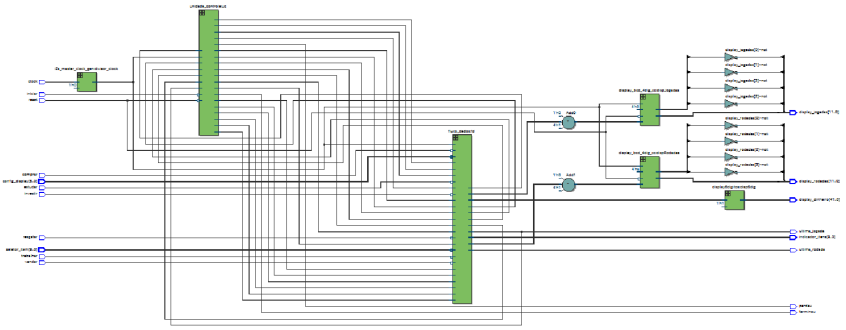


Imagem 7: RTL Viewer do circuito principal

1.2.1 Projeto do Fluxo de Dados

O fluxo de dados é o núcleo operacional do sistema, projetado para processar e armazenar todas as variáveis financeiras através de uma arquitetura de 21 bits em complemento de dois, o que permite lidar com valores negativos para representar dívidas ou saldos devedores. No centro desta estrutura estão cinco registradores principais que mantêm o estado atual do saldo, do salário, do valor investido e dos gastos (fixos e únicos), além de um registrador de 10 bits dedicado ao inventário de itens adquiridos. A manipulação desses dados ocorre por meio de dois módulos especializados: o

processador de ações e o processador de eventos. O primeiro traduz comandos diretos do utilizador, como "Estudar" (que reduz o saldo e aumenta o potencial de salário) ou "Trabalhar" (que soma o salário ao saldo), enquanto o segundo gere dinâmicas automáticas de tempo, como a aplicação de rendimentos de 25% sobre investimentos e a dedução de despesas fixas ao final de cada período. Para a gestão temporal, o fluxo de dados incorpora contadores de módulo-N que dividem o progresso em jogadas e rodadas, simulando a passagem de quadrimestres e anos. A saída de dados é gerida por um multiplexador que, orientado pela configuração do utilizador, seleciona qual métrica financeira deve ser encaminhada para a lógica de visualização nos displays.

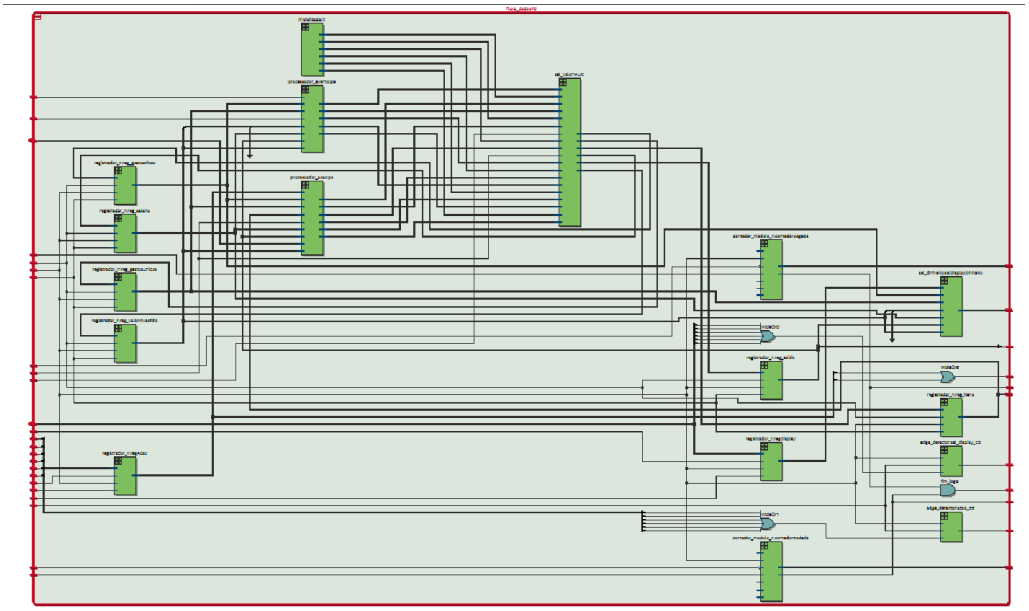


Imagem 8: RTL Viewer do Fluxo de Dados

Verilog do Fluxo de dados:

```
module fluxo_dados (
    input clock,
    input estudar,
    input trabalhar,
    input investir,
    input resgatar,
    input comprar,
    input vender,
    input [5:0] config_display,
    input [9:0] seletor_item,

    input rstED,
```

```
    input init,
    input processaE,
    input rende,
    input despesa,
    input zeraCJ,
    input contaCJ,
    input zeraCR,
    input contaCR,
    input zeraD,
    input registraD,
    input zeraA,
    input registraA,
    input zeraV,
    input registraV,
    input zeraM,
    input registraM,

    output fim_jogo,
    output fim_perdeu,
    output fim_rodada,
    output acao_pulso,
    output eh_jogada,
    output display_pulso,
    output [3:0] contagem, //
    output [3:0] rodada, //

    output ultima_rodada,
    output [23:0] dinheiro,
    output [9:0] itens_out
);

    //em complemento de 2 -> bit 21 = 1 indica negativo
    wire [20:0] saldo;
    wire [20:0] saldo_next;
    wire [20:0] salario;
    wire [20:0] salario_next;
    wire [20:0] valorInvestido;
    wire [20:0] valorInvestido_next;
    wire [20:0] rendimento;
```

```
    wire [20:0] rendimento_next;
    wire [20:0] gastosFixos;
    wire [20:0] gastosFixos_next;
    wire [20:0] gastosUnicos;
    wire [20:0] gastosUnicos_next;
    wire [9:0] itens;
    wire [9:0] itens_next;

    wire [5:0] acoes_out;

    assign itens_out = itens;

//////////////display///////////////////////////////
    wire sel_display;
    wire [5:0] config_display_out;
    wire [2:0] addr_read;

    edge_detector sel_display_ED (
        .clock ( clock ),
        .reset ( rstED ),
        .sinal ( sel_display ),
        .pulso ( display_pulso )
    );
    assign sel_display = |config_display;

    // registrador da opcao de selecao do display
    registrador_n #(.N(6)) regDisplay (
        .clock  ( clock ),
        .clear  ( zeraD ),
        .enable ( registraD ),
        .D      ( config_display ),
        .Q      ( config_display_out )
    );

    /*// transformador de formato de config_display (botoes) p/ endereco de leitura
    encoder #(.N(6)) addr_display (
        .entrada ( config_display_out ),
        .saida ( addr_read )
    );*/
```

```
    sel_dinheiro selDisplayDinheiro (
        .saldo          ( saldo ),
        .salario        ( salario ),
        .valorInvestido ( valorInvestido ),
        .rendimento     ( rendimento ),
        .gastosFixos    ( gastosFixos ),
        .gastosUnicos   ( gastosUnicos ),
        .config_display ( config_display_out ),
        .dinheiro       ( dinheiro )
    );

//////////info////////////////////////////////////////////////////////////////

    registrador_n #(.N(21)) reg_saldo (
        .clock  ( clock ),
        .clear  ( zeraV ),
        .enable ( registraV ),
        .D      ( saldo_next ),
        .Q      ( saldo )
    );
    registrador_n #(.N(21)) reg_salario (
        .clock  ( clock ),
        .clear  ( zeraV ),
        .enable ( registraV ),
        .D      ( salario_next ),
        .Q      ( salario )
    );
    registrador_n #(.N(21)) reg_valorInvestido (
        .clock  ( clock ),
        .clear  ( zeraV ),
        .enable ( registraV ),
        .D      ( valorInvestido_next ),
        .Q      ( valorInvestido )
    );
    /*registrador_n #(.N(21)) reg_rendimento (
        .clock  ( clock ),
        .clear  ( zeraV ),
```

```
    .enable ( registraV ),
    .D      ( rendimento_next ),
    .Q      ( rendimento )
);*/
assign rendimento = valorInvestido >> 2; //25% ao ano (rodada)
registrador_n #(.N(21)) reg_gastosFixos (
    .clock  ( clock ),
    .clear  ( zeraV ),
    .enable ( registraV ),
    .D      ( gastosFixos_next ),
    .Q      ( gastosFixos )
);
registrador_n #(.N(21)) reg_gastosUnicos (
    .clock  ( clock ),
    .clear  ( zeraV ),
    .enable ( registraV ),
    .D      ( gastosUnicos_next ),
    .Q      ( gastosUnicos )
);

registrador_n #(.N(10)) reg_itens (
    .clock  ( clock ),
    .clear  ( zeraV ),
    .enable ( registraV ),
    .D      ( itens_next ),
    .Q      ( itens )
);

////////////////////////////////////////////////////////////////////////////
////////////////processamento/////////////////////////////////////////////////

wire [20:0] saldo_proc;
wire [20:0] salario_proc;
wire [20:0] valorInvestido_proc;
wire [20:0] gastosFixos_proc;
wire [20:0] gastosUnicos_proc;
wire [9:0]  itens_proc;
processador_acao pa (
    .saldo_in           ( saldo ),
```

```
    .salario_in         ( salario ),
    .valorInvestido_in   ( valorInvestido ),
    .gastosFixos_in      ( gastosFixos ),
    .gastosUnicos_in     ( gastosUnicos ),
    .acao               ( acoes_out ),
    .processaE          ( processaE ),
    .seletor_item       ( seletor_item ),
    .itens_in           ( itens ),
    .saldo_out          ( saldo_proc ),
    .salario_out        ( salario_proc ),
    .valorInvestido_out  ( valorInvestido_proc ),
    .gastosFixos_out     ( gastosFixos_proc ),
    .gastosUnicos_out    ( gastosUnicos_proc ),
    .itens_out          ( itens_proc )
);

wire [20:0] saldo_evento;
wire [20:0] salario_evento;
wire [20:0] valorInvestido_evento;
wire [20:0] gastosFixos_evento;
wire [20:0] gastosUnicos_evento;
processador_evento pe (
    .saldo_in           ( saldo ),
    .salario_in         ( salario ),
    .valorInvestido_in   ( valorInvestido ),
    .rendimento_in       ( rendimento ),
    .gastosFixos_in      ( gastosFixos ),
    .gastosUnicos_in     ( gastosUnicos ),
    .rende              ( rende ),
    .despesa            ( despesa ),
    .saldo_out          ( saldo_evento ),
    .salario_out        ( salario_evento ),
    .valorInvestido_out  ( valorInvestido_evento ),
    .gastosFixos_out     ( gastosFixos_evento ),
    .gastosUnicos_out    ( gastosUnicos_evento )
);

wire [20:0] saldo_init;
wire [20:0] salario_init;
```

```
wire [20:0] valorInvestido_init;
wire [20:0] gastosFixos_init;
wire [20:0] gastosUnicos_init;
wire [9:0]  itens_init;
inicializador i (
    .saldo          ( saldo_init ),
    .salario        ( salario_init ),
    .valorInvestido ( valorInvestido_init ),
    .gastosFixos    ( gastosFixos_init ),
    .gastosUnicos   ( gastosUnicos_init ),
    .itens          ( itens_init )
);

sel_valor MUX (
    .saldo_init           ( saldo_init ),
    .salario_init         ( salario_init ),
    .valorInvestido_init  ( valorInvestido_init ),
    .gastosFixos_init     ( gastosFixos_init ),
    .gastosUnicos_init    ( gastosUnicos_init ),
    .itens_init           ( itens_init ),
    .saldo_proc           ( saldo_proc ),
    .salario_proc         ( salario_proc ),
    .valorInvestido_proc  ( valorInvestido_proc ),
    .gastosFixos_proc     ( gastosFixos_proc ),
    .gastosUnicos_proc    ( gastosUnicos_proc ),
    .itens_proc           ( itens_proc ),
    .saldo_evento         ( saldo_evento ),
    .salario_evento       ( salario_evento ),
    .valorInvestido_evento ( valorInvestido_evento ),
    .gastosFixos_evento   ( gastosFixos_evento ),
    .gastosUnicos_evento  ( gastosUnicos_evento ),
    .init                 ( init ),
    .processaE            ( processaE ),
    .saldo_out            ( saldo_next ),
    .salario_out          ( salario_next ),
    .valorInvestido_out   ( valorInvestido_next ),
    .gastosFixos_out      ( gastosFixos_next ),
    .gastosUnicos_out     ( gastosUnicos_next ),
    .itens_out            ( itens_next )
```

```
    );

////////////////////////////////////////////////////////////////
////////////////acao////////////////////////////////////////////

    wire [5:0] acoes;
    wire tem_acao;

    edge_detector acao_ED (
        .clock ( clock ),
        .reset ( rstED ),
        .sinal ( tem_acao ),
        .pulso ( acao_pulso )
    );
    assign acoes = {estudar, trabalhar, investir, resgatar, comprar, vender};
    assign tem_acao = |acoes;


    // registrador da acao realizada
    registrador_n #(.N(6)) regAcao (
        .clock  ( clock ),
        .clear  ( zeraA ),
        .enable ( registraA ),
        .D      ( acoes ),
        .Q      ( acoes_out )
    );


////////////////////////////////////////////////////////////////////////////
//////////////jogada////////////////////////////////////////////////////////
    wire [1:0] jogada;
    assign jogada = {acoes_out[5], acoes_out[4]};
    assign eh_jogada = |jogada;



    // contador_modulo_n contadorJogada
    contador_modulo_n #(.N(4)) contadorJogada (
      .clock  ( clock ),
```

```
    .clr    ( ~zeraCJ ),
    .ld     ( 1'b1 ),
    .ent    ( 1'b1 ),
    .enp    ( contaCJ ),
    .modulo ( 4'b0010 ),  //(modulo +1) jogadas, 3 jogadas -> 3 quadris por rodada (ano)
    .D      ( 4'b0000 ),
    .Q      ( contagem ),
    .rco    ( fim_rodada )
  );
  // 1 jogada -> 1 quadrimestre



  // contador_modulo_n contadorRodada
  contador_modulo_n #(.N(4)) contadorRodada (
    .clock  ( clock ),
    .clr    ( ~zeraCR ),
    .ld     ( 1'b1 ),
    .ent    ( 1'b1 ),
    .enp    ( contaCR ),
    .modulo ( 4'b1110),  //(modulo + 1) rodadas, 15 rodadas -> 15 trimestres no jogo
    .D      ( 4'b0000 ),
    .Q      ( rodada ),
    .rco    ( ultima_rodada )
  );

  assign fim_jogo = fim_rodada && ultima_rodada;
  assign fim_perdeu = saldo[20];

endmodule
```

1.2.2 Projeto da Unidade de Controle

A unidade de controle atua como o cérebro do sistema, implementada como uma Máquina de Estados Finita (FSM) do tipo Moore. O seu funcionamento é ditado por uma sequência de estados que garantem a ordem lógica das operações e a integridade das regras do jogo. Ao ser iniciado, o sistema passa pelo estado de inicialização, onde define os valores base de saldo e salário, movendo-se depois para um estado de espera ativa.

Neste ponto, a unidade monitoriza entradas do utilizador, como o pulso de ação ou de display. Quando uma jogada é detectada, a FSM transita para estados de registo e processamento, ativando sinais de controle específicos que habilitam a escrita nos registradores do fluxo de dados. Após cada alteração, a unidade de controle avalia automaticamente as condições de terminação através de estados de verificação, determinando se o jogador faliu (saldo negativo) ou se atingiu o objetivo final. A transição entre rodadas também é coordenada por esta unidade, que sinaliza o momento exato para o cálculo de dividendos e o incremento dos contadores de tempo, assegurando que o fluxo de dados apenas atualize as suas informações em momentos sincronizados com o clock do sistema.

Nesta semana, os sinais zeraCJ e zeraCR foram adicionados ao estado Start, e um novo estado AplicarDespesa foi criado entre ProcessarAcao e VerificaPerdeu, com a responsabilidade de aplicar as despesas antes de verificar se o saldo é negativo.

Tabela 5 – Descrição da Unidade de Controle do Sistema

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados
inicial	zera registradores e inicia valores na unidade de processamento	standby	-
standby	registra valores iniciais	preparacao	-
preparacao	zera contadores	espera / preparacao	preparacao deve durar dois ciclos
espera	espera seleção de botão (configuração display ou ação)	Espera / registra Display / registra Acao	se for identificada uma ação pelo jogador, vai para registraAcao; se for identificada uma seleção de configuração de display, vai para registraDisplay; caso contrário, continua esperando
registra Display	registra a configuração de display escolhida	espera	-
registra Acao	registra a ação escolhida pelo jogador	processar Acao	-
processar Acao	processa os valores de acordo com a ação escolhida	aplicar Despesa	-
aplicar Despesa	aplica despesa	verifica Perdeu	-
verifica Perdeu	verifica se o saldo é negativo (perdeu)	fimPerdeu / jogadaOu Acao	se o saldo é negativo, finaliza o jogo; se não for, continua para verificar se é jogada ou ação
jogadaOu Acao	verifica se é jogada (incrementa os contadores) ou se não é	espera / verificaFim	se for jogada, continua para incrementar os contadores; se não for, volta para espera

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados
verificaFim	verifica se o jogo acabou (última jogada da última rodada)	fim / verificaFim Rodada	se o sinal fim_jogo estiver ativo, finaliza o jogo (no estado fim); se não, verifica o fim da rodada
verificaFim Rodada	decide qual contador deve ser incrementado após verificar a última jogada de uma rodada	nova Rodada / proxima Jogada	se o sinal fim_rodada estiver ativo, vai para novaRodada; se não estiver, vai para proximaJogada
nova Rodada	incrementa contador de rodada, zera o de jogada	espera	-
proxima Jogada	incrementa contador de jogada	espera	-
fimPerdeu	sinaliza informações finais do jogador, inclusive o sinal perdeu	preparacao, RegistraDisplayFP	-
RegistraDisplayFP	estado para continuar mostrando o display mesmo após o fim do jogo	fimPerdeu	-
fim	sinaliza informações finais do jogador, inclusive o sinal terminou	preparacao	-
RegistraDisplayF	estado para continuar mostrando o display mesmo após o fim do jogo	fim	

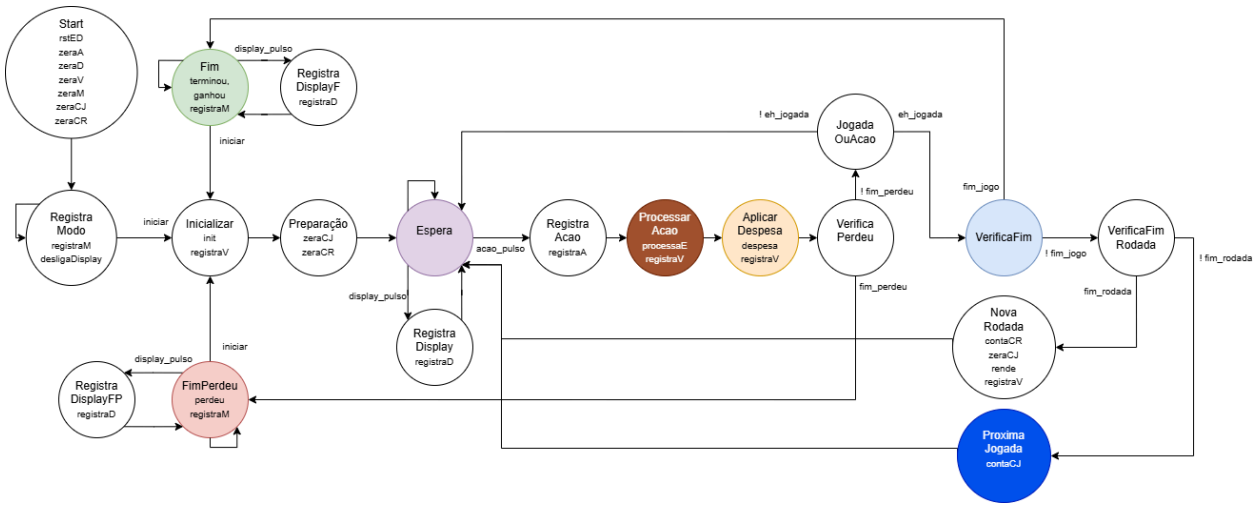


Imagem 9: Diagrama de Transição de Estados

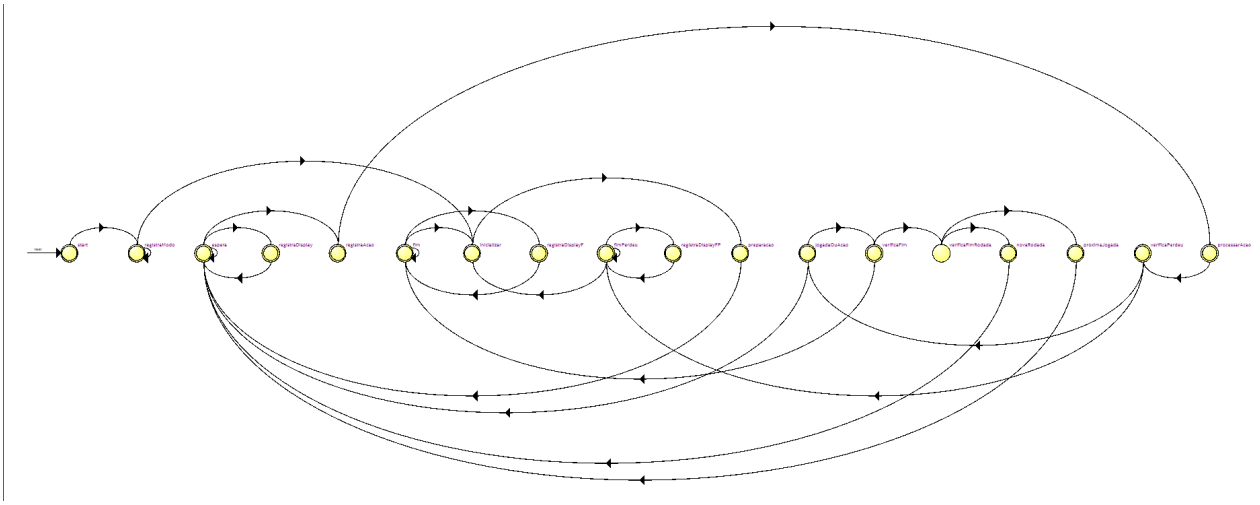


Imagem 10: State Machine no RTL Viewer

Verilog da Unidade de Controle:

```
module unidade_controle (
    input       clock,
    input       reset,
    input       iniciar,
    input       acao_pulso,
    input       eh_jogada,
    input       display_pulso,
    input       fim_jogo,
    input       fim_perdeu,
    input       fim_rodada,

    output reg rstED,
    output reg init,
    output reg processaE,
    output reg rende,
    output reg despesa,
    output reg zeraCJ,
    output reg contaCJ,
    output reg zeraCR,
    output reg contaCR,
    output reg zeraD,
    output reg registraD,
    output reg zeraA,
    output reg registraA,
    output reg zeraV,
    output reg registraV,
    output reg zeraM,
    output reg registraM,

    output reg terminou,
    output reg perdeu,
    output reg [4:0] estado
);

    // Define estados
    parameter start             = 5'b00000;  // 0
    parameter registraModo      = 5'b00001;  // 1
    parameter inicializar       = 5'b00010;  // 2
    parameter preparacao        = 5'b00011;  // 3
    parameter espera            = 5'b00100;  // 4
```

```
parameter registraAcao      = 5'b00101;  // 5
parameter processarAcao     = 5'b00110;  // 6
parameter verificaPerdeu    = 5'b00111;  // 7
parameter jogadaOuAcao      = 5'b01000;  // 8
parameter verificaFim       = 5'b01001;  // 9
parameter verificaFimRodada = 5'b01010;  // A
parameter proximaJogada     = 5'b01011;  // B
parameter novaRodada        = 5'b01100;  // C

parameter registraDisplay   = 5'b11010;
parameter registraDisplayF  = 5'b11011;
parameter registraDisplayFP = 5'b11100;
parameter fim               = 5'b11101;
parameter fimPerdeu         = 5'b11110;

// Variaveis de estado
reg [4:0] Eatual, Eprox;
reg prep_count;

// contador para estados que duram mais de 1 ciclo de clock
always @(posedge clock or posedge reset) begin
    if (reset) begin
        Eatual <= start;
        prep_count <= 0;
    end
    else begin
        Eatual <= Eprox;

        // Contador ativo no estado preparacao
        if (Eatual == preparacao)
            prep_count <= prep_count + 1;
        else
            prep_count <= 0;
    end
end

// Logica de proximo estado
always @* begin
    Eprox = Eatual;
    case (Eatual)
        start:          Eprox = registraModo;
        registraModo:   Eprox = (iniciar) ? inicializar : registraModo;
```

```
        inicializar:      Eprox = preparacao;
        preparacao: /*begin
                    if (prep_count == 1)
                        Eprox = espera;
                    else
                        Eprox = preparacao;
                end */
                        Eprox = espera;
        espera:           Eprox = (acao_pulso) ? registraAcao : ((display_pulso) ? registraDisplay :
espera);
        registraAcao:     Eprox = processarAcao;
        processarAcao:    Eprox = verificaPerdeu;
        verificaPerdeu:   Eprox = (fim_perdeu) ? fimPerdeu : jogadaOuAcao;
        jogadaOuAcao:     Eprox = (eh_jogada) ? verificaFim : espera;
        verificaFim:      Eprox = (fim_jogo) ? fim : verificaFimRodada;
        verificaFimRodada: Eprox = (fim_rodada) ? novaRodada : proximaJogada;
        proximaJogada:    Eprox = espera;
        novaRodada:       Eprox = espera;
        registraDisplay:  Eprox = espera;
        registraDisplayF:  Eprox = fim;
        registraDisplayFP: Eprox = fimPerdeu;
        fim:              Eprox = (iniciar) ? inicializar : ((display_pulso) ? registraDisplayF : fim);
        fimPerdeu:        Eprox = (iniciar) ? inicializar : ((display_pulso) ? registraDisplayFP : fimPerdeu);

        default:          Eprox = start;
    endcase
end

// Logica de saida (maquina Moore)
always @* begin
    rstED          = (Eatual == start) ? 1'b1 : 1'b0;
    init           = (Eatual == inicializar) ? 1'b1 : 1'b0;
    processaE      = (Eatual == processarAcao) ? 1'b1 : 1'b0;
    zeraCJ         = (Eatual == preparacao || Eatual == novaRodada) ? 1'b1 : 1'b0;
    rende          = (Eatual == novaRodada) ? 1'b1 : 1'b0;
    despesa        = (Eatual == proximaJogada || Eatual == novaRodada) ? 1'b1 : 1'b0;
    contaCJ        = (Eatual == proximaJogada) ? 1'b1 : 1'b0;
    zeraCR         = (Eatual == preparacao) ? 1'b1 : 1'b0;
    contaCR        = (Eatual == novaRodada) ? 1'b1 : 1'b0;
    zeraD          = (Eatual == start) ? 1'b1 : 1'b0;
    registraD      = (Eatual == registraDisplay || Eatual == registraDisplayF || Eatual ==
registraDisplayFP) ? 1'b1 : 1'b0;
    zeraA          = (Eatual == start) ? 1'b1 : 1'b0;
```

```
        registraA       = (Eatual == registraAcao) ? 1'b1 : 1'b0;
        zeraV           = (Eatual == start) ? 1'b1 : 1'b0;
        registraV       = (Eatual == inicializar || Eatual == processarAcao || Eatual == proximaJogada ||
Eatual == novaRodada) ? 1'b1 : 1'b0;
        zeraM           = (Eatual == start) ? 1'b1 : 1'b0;
        registraM       = (Eatual == registraModo || Eatual == fim || Eatual == fimPerdeu) ? 1'b1 : 1'b0;
        /*registraV[0]    =
        registraV[1]    =
        registraV[2]    =
        registraV[3]    =
        registraV[4]    =
        registraV[5]    =*/
        terminou        = (Eatual == fim) ? 1'b1 : 1'b0;
        perdeu          = (Eatual == fimPerdeu) ? 1'b1 : 1'b0;

        // Saida de depuracao (estado)
        estado          = Eatual;
    end

endmodule
```

1.2.3 Projeto do Sistema Digital Programável

A integração entre o fluxo de dados e a unidade de controle num sistema digital programável é o que permite que o jogo funcione como uma entidade coesa. Esta ligação é feita através de um barramento de sinais de controle e de status. A unidade de controle envia comandos unidirecionais que funcionam como permissões de escrita (habilitação de carga) e sinais de seleção para os módulos aritméticos. Em contrapartida, o fluxo de dados devolve sinais de status, como indicadores de fim de jogo ou de rodada, que servem como variáveis de entrada para a lógica de próxima transição de estado da FSM. Todo este ecossistema é encapsulado num módulo de topo que faz a ponte com o hardware físico, mapeando botões e interruptores para as entradas lógicas e conectando as saídas de dados a descodificadores de sete segmentos. A estabilidade desta integração é garantida por um sinal de clock ajustado, que sincroniza a resposta rápida da lógica digital com a percepção humana, permitindo que cada decisão financeira do jogador seja processada, validada e exibida em tempo real.

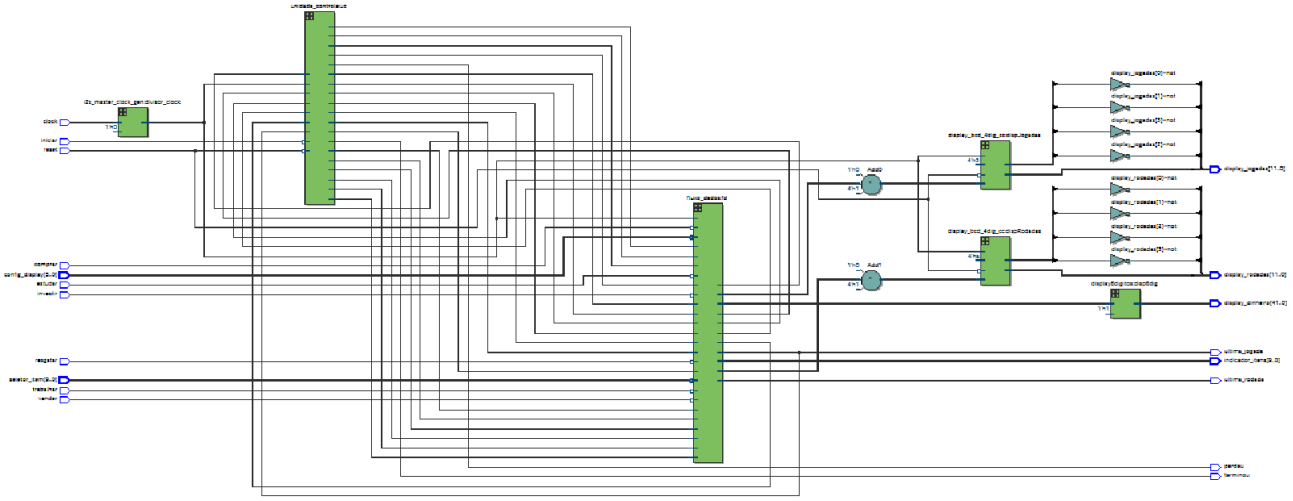


Imagem 11: RTL Viewer do projeto completo

Verilog do Circuito:

```
module circuito_jogo_financeiro (
    input clock,
    input reset,
    input iniciar,
    input estudar,
    input trabalhar,
    input investir,
    input resgatar,
    input comprar,
    input vender,
    input [9:0] seletor_item,
    input [5:0] config_display,

    //output [4:0] estado, //

    output [11:0] display_jogadas,
    output [11:0] display_rodadas,
    // A, B, C, D, E, F, G, DP, D1, D2, D3, D4

    output [41:0] display_dinheiro,
```

```
    output [9:0] indicador_itens,
    output ultima_jogada,
    output ultima_rodada,
    output terminou,
    output perdeu
);


    wire w_fim_jogo;
    wire w_fim_perdeu;
    wire w_fim_rodada;

    assign ultima_jogada = w_fim_rodada;
    wire [23:0] dinheiro;

    wire w_rstED;
    wire w_init;
    wire w_processaE;
    wire w_rende;
    wire w_despesa;
    wire w_zeraCJ;
    wire w_contaCJ;
    wire w_zeraCR;
    wire w_contaCR;
    wire w_zeraD;
    wire w_registraD;
    wire w_zeraA;
    wire w_registraA;
    wire w_zeraV;
    wire w_registraV;
    wire w_zeraM;
    wire w_registraM;

    wire w_acao_pulso;
    wire w_eh_jogada;
```

```
    wire w_display_pulso;
    wire [3:0] w_contagem;
    wire [3:0] w_rodada;
    wire [4:0] w_estado;

 wire clock_ajustado;

    //assign estado = w_estado;

 i2s_master_clock_gen  #(.CLOCK_DIV(781)) divisor_clock(
    .clk (clock),
    .rst(1'b0),
    .sck_o(),
    .ws_o (clock_ajustado)
);

    fluxo_dados fd (
        .clock          ( clock_ajustado ),
        .estudar        ( ~estudar ),
        .trabalhar      ( ~trabalhar ),
        .investir       ( ~investir ),
        .resgatar       ( ~resgatar ),
        .comprar        ( ~comprar ),
        .vender         ( ~vender ),
        .config_display ( ~config_display ),
        .seletor_item   ( ~seletor_item ),

        .rstED          ( w_rstED ),
        .init           ( w_init ),
        .processaE      ( w_processaE ),
        .rende          ( w_rende ),
        .despesa        ( w_despesa ),
        .zeraCJ         ( w_zeraCJ ),
        .contaCJ        ( w_contaCJ ),
        .zeraCR         ( w_zeraCR ),
```

```
    .contaCR        ( w_contaCR ),
    .zeraD          ( w_zeraD ),
    .registraD      ( w_registraD ),
    .zeraA          ( w_zeraA ),
    .registraA      ( w_registraA ),
    .zeraV          ( w_zeraV ),
    .registraV      ( w_registraV ),
    .zeraM          ( w_zeraM ),
    .registraM      ( w_registraM ),

    .fim_jogo       ( w_fim_jogo ),
    .fim_perdeu     ( w_fim_perdeu ),
    .fim_rodada     ( w_fim_rodada ),
    .acao_pulso     ( w_acao_pulso ),
    .eh_jogada      ( w_eh_jogada ),
    .display_pulso  ( w_display_pulso ),
    .contagem       ( w_contagem ),
    .rodada         ( w_rodada ),

    .ultima_rodada  ( ultima_rodada ),
    .dinheiro       ( dinheiro ),
    .itens_out      ( indicador_itens )
);

unidade_controle uc (
    .clock          ( clock_ajustado ),
    .reset          ( ~reset ),
    .iniciar        ( ~iniciar ),
    .acao_pulso     ( w_acao_pulso ),
    .eh_jogada      ( w_eh_jogada ),
    .display_pulso ( w_display_pulso ),
    .fim_jogo       ( w_fim_jogo ),
    .fim_perdeu     ( w_fim_perdeu ),
    .fim_rodada     ( w_fim_rodada ),
```

```
    .rstED        ( w_rstED ),
    .init         ( w_init ),
    .processaE    ( w_processaE ),
    .rende        ( w_rende ),
    .despesa      ( w_despesa ),
    .zeraCJ       ( w_zeraCJ ),
    .contaCJ      ( w_contaCJ ),
    .zeraCR       ( w_zeraCR ),
    .contaCR      ( w_contaCR ),
    .zeraD        ( w_zeraD ),
    .registraD    ( w_registraD ),
    .zeraA        ( w_zeraA ),
    .registraA    ( w_registraA ),
    .zeraV        ( w_zeraV ),
    .registraV    ( w_registraV ),
    .zeraM        ( w_zeraM ),
    .registraM    ( w_registraM ),
    .terminou     ( terminou ),
    .perdeu       ( perdeu ),
    .estado       ( w_estado )
);


display6digitos disp6dig (
    .dinheiro ( dinheiro ),
    .enable ( 1'b1 ),
    .display ( display_dinheiro )
);

wire [7:0] seg_jogada;
wire [3:0] dig_jogada;
wire [7:0] seg_rodada;
wire [3:0] dig_rodada;
assign display_jogadas = { seg_jogada, ~dig_jogada };
assign display_rodadas = { seg_rodada, ~dig_rodada };
```

```
    // --- Display de JOGADAS ---
    display_bcd_4dig_cc dispJogadas (
        .clock  ( clock_ajustado ),
        .reset  ( ~reset ),
        .valor  ( w_contagem+4'd1 ),
        .limite ( 4'd3 ),       // Limite de 3 jogadas por rodada
        .seg    ( seg_jogada ), // Conectar aos pinos A-DP do display 1 (ativo alto)
        .dig    ( dig_jogada )  // Conectar aos pinos D1-D4 do display 1 (ativo baixo)
    );

    // --- Display de RODADAS ---
    display_bcd_4dig_cc dispRodadas (
        .clock  ( clock_ajustado ),
        .reset  ( ~reset ),
        .valor  ( w_rodada+4'd1 ),
        .limite ( 4'd10 ),      // Total de 10 rodadas
        .seg    ( seg_rodada ), // Conectar aos pinos A-DP do display 1 (ativo alto)
        .dig    ( dig_rodada )  // Conectar aos pinos D1-D4 do display 1 (ativo baixo)
    );

endmodule
```

1.3 Plano de Testes do Hardware Programável e Simulações

Este capítulo tem o objetivo de documentar a estratégia dos testes do sistema desenvolvido e documentado na seção 3.2 e apresentar os resultados das respectivas simulações realizadas.

1.3.1 Cenário de Teste 1 – Teste do funcionamento total do jogo

Por ser a primeira semana, os testes realizados devem contemplar todas as funções do projeto do labdig 1, de forma a deixá-lo completamente funcional antes de começar as mudanças de transmissão serial.

Tabela – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Simulado OK?	Análise de Não Conformidades
Teste físico	Displays funcionais	Sim	-
Iniciar o jogo	Displays resetados e jogo iniciado	Sim	-
Comprar várias vezes	O jogador deve perder com saldo negativo	Sim	-
Estudar várias vezes	O jogador deve ter seu salário aumentado e o seu saldo diminuído	Sim	-
Tentar comprar o mesmo item duas vezes	O jogo não deve permitir	Sim	-
Fazer uma jogada qualquer	O display de jogada deve mudar e o display de rodada deve mudar se essa jogada foi a última da rodada	Sim	-
Resetar o jogo	Tudo deve ser resetado, e o jogador não deve ser capaz de jogar o jogo	Sim	-
Vender um item que não se comprou	O jogo não deve permitir	Sim	-
Selecionar dois itens ao mesmo tempo para comprar	O jogo não deve permitir	Sim	-

1.4 Detalhamento do Projeto de Software

Na Semana 1, as atividades práticas concentram-se exclusivamente no hardware físico e na caracterização do módulo leitor de RFID (RC522), sem desenvolvimento de software. O foco consiste em mapear as conexões de alimentação e barramento SPI, verificar os níveis de tensão junto à placa FPGA DE0-CV e analisar os sinais elétricos do componente na bancada com auxílio do Analog Discovery, garantindo a integridade da leitura das tags antes de qualquer implementação lógica ou de telemetria.

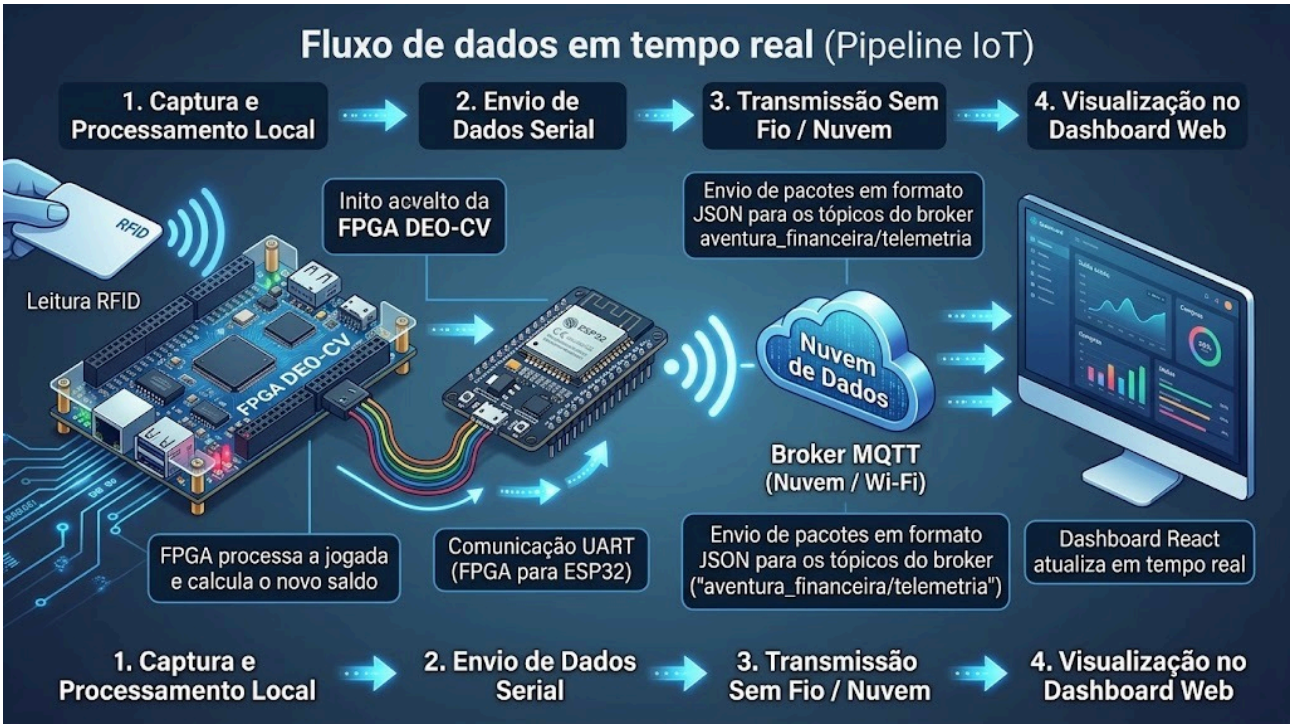


Imagem 12: Fluxo de dados em tempo real (Pipeline IoT)

1.5 Prototipação do Projeto

Este capítulo tem o objetivo de documentar a construção das partes físicas do projeto físico do projeto desenvolvidas na Semana 1.

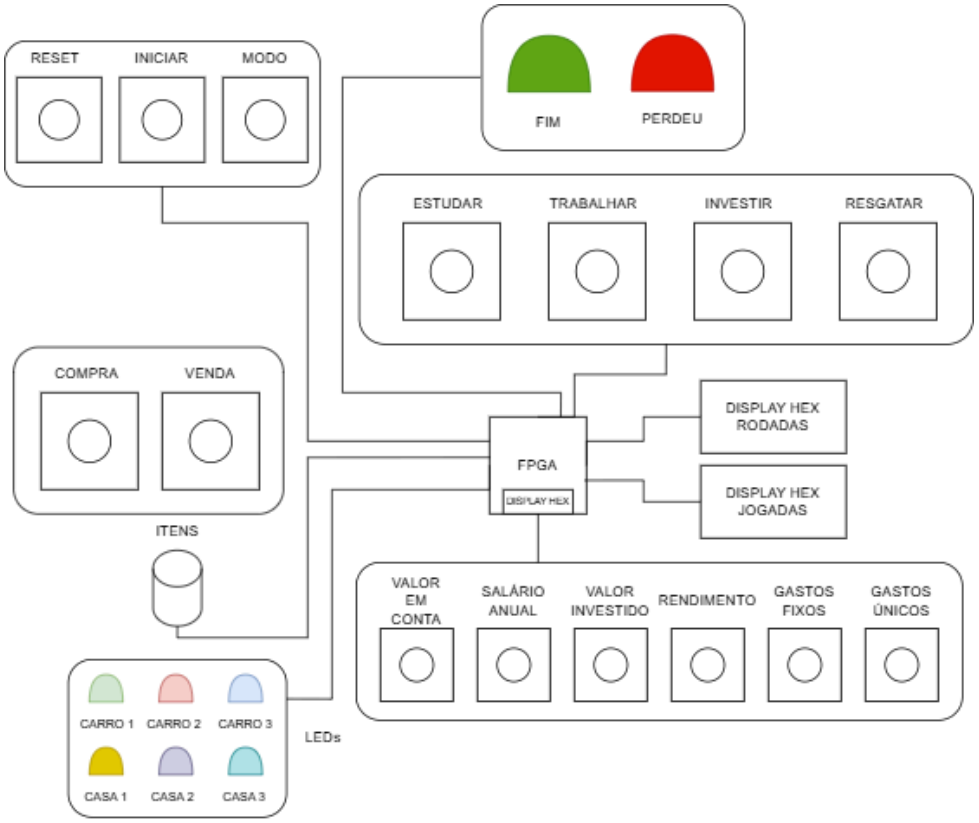
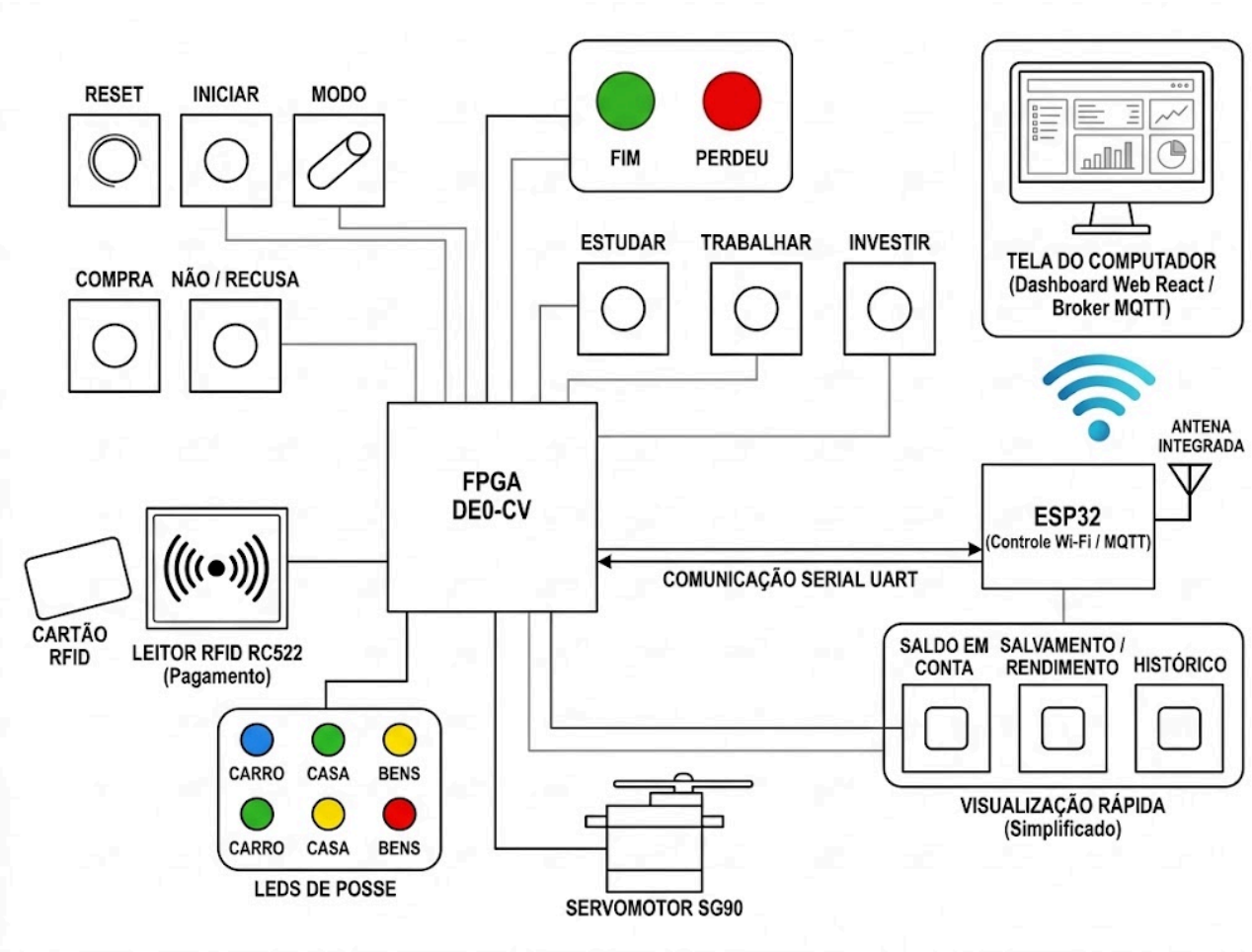


Imagem 13: Esquemático dos componentes periféricos e FPGA



1.6 Implantação do Projeto

Esta seção tem o objetivo de documentar as atividades práticas de execução do projeto desenvolvido e documentado no ambiente do Laboratório de Projeto de Sistemas Digitais.

1.6.1 Pinagem da Placa FPGA

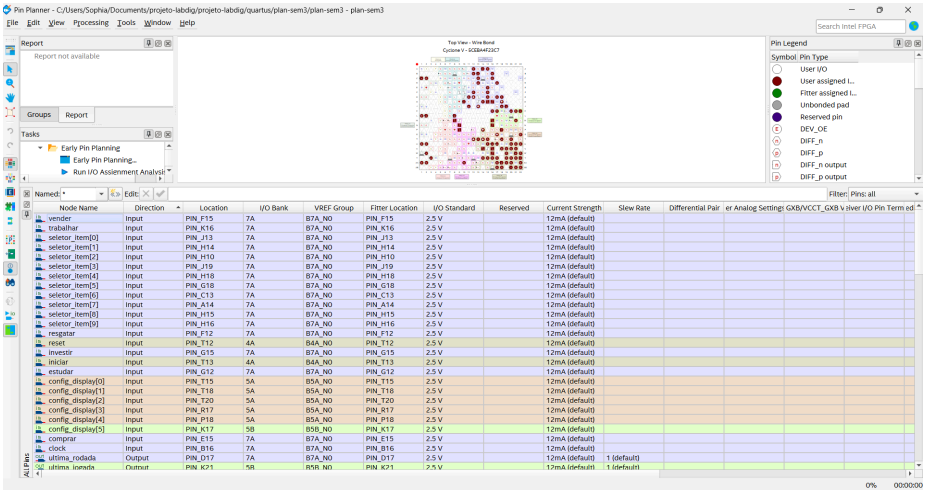


Imagem 14: Pin Planner parte 1

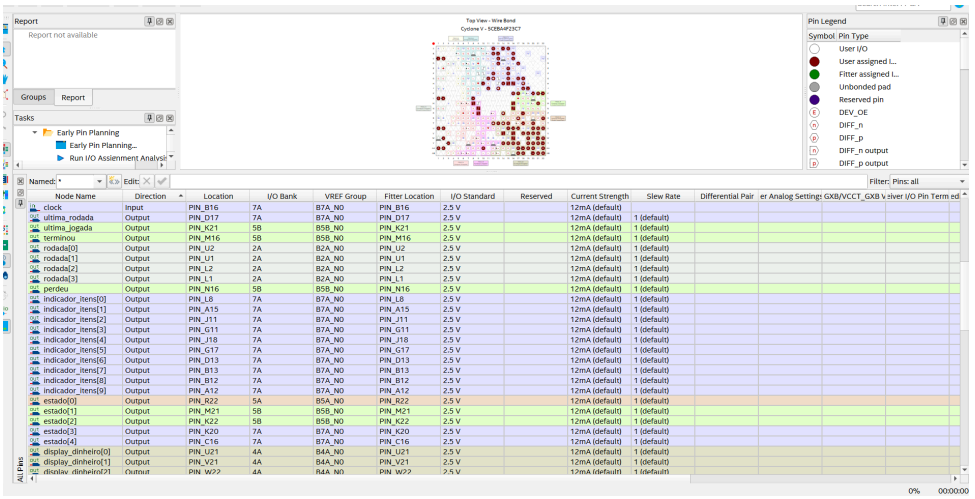


Imagem 15: Pin Planner parte 2

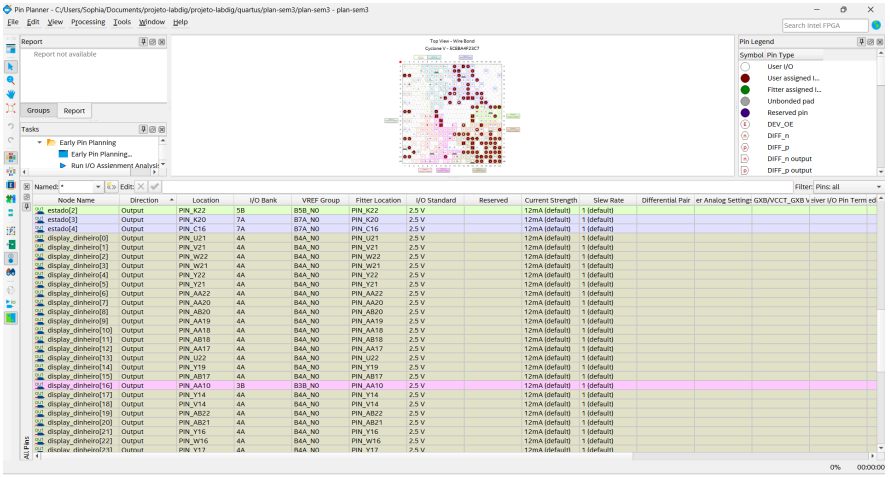


Imagem 16: Pin Planner parte 3

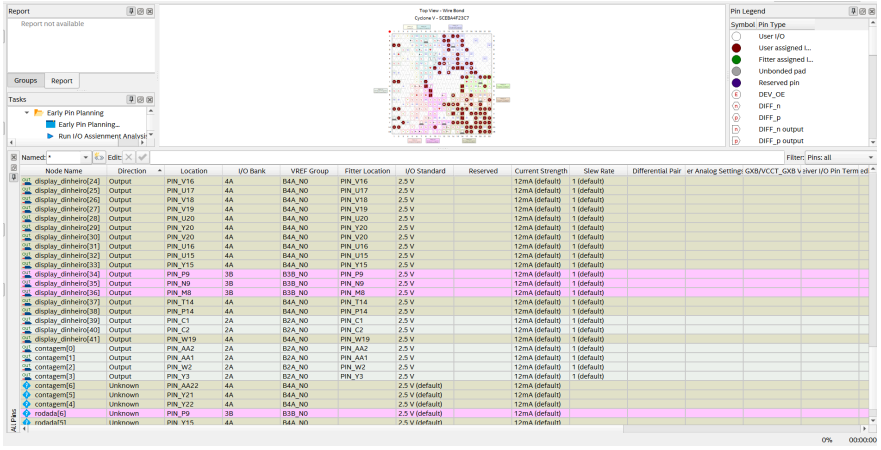


Imagem 17: Pin Planner parte 4

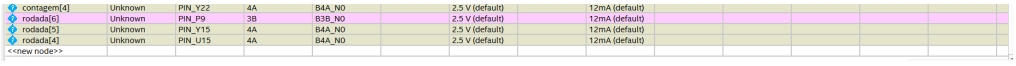


Imagem 18: Pin Planner parte 5

https://www.blogdarobotica.com/2022/06/30/como-utilizar-display-de-7-segmentos-com-4-digitos-catodo-comum-com-arduino/

https://i.ytimg.com/vi/fYAIE1u5rno/maxresdefault.jpg

Tabela: Pinagem da Placa FPGA

Tipo	Sinal	Pino na Placa DE0-CV	Pino no FPGA	Analog Discovery
entradas	CLOCK	GPIO_0_D10	PIN_N21	Patterns – Clock – 1 KHz StaticIO – LED – DIO15
	INICIAR	GPIO_1_D23	PIN_E14	-
	RESET	GPIO_1_D21	PIN_B15	-
	TRABALHAR	BOTÃO AZUL GPIO_1_D35	PIN_K16	
	ESTUDAR	BOTÃO BRANCO GPIO_1_D33	PIN_G12	
	INVESTIR	BOTÃO PRETO GPIO_1_D31	PIN_G15	
	RESGATAR	BOTÃO VERMELHO GPIO_1_D29	PIN_F12	
	VENDER	BOTÃO AMARELO GPIO_1_D27	PIN_F15	
	COMPRAR	BOTÃO VERD E GPIO_1_D25	PIN_E16	
saídas	PERDEU	GPIO_0_D0	PIN_N16	-
	TERMINOU	GPIO_0_D2	PIN_M16	-
	ultima_rodada	GPIO_0_D4	PIN_D17	-
	ultima_jogada	GPIO_0_D6	PIN_K21	-
	dinheiro_0 [6:0]	Display HEX0	PIN_U21, V21, W22, W21, Y22, Y21, AA22	-
	dinheiro_1 [13:7]	Display HEX1	PIN AA20, AB20, AA19, AA18, AB18, AA17, U22	-
	dinheiro_2 [20:14]	Display HEX2	PIN_Y19, AB17, AA10, Y14, V14, AB22, AB21	-
	dinheiro_3 [27:21]	Display HEX3	PIN Y16, W16, Y17, V16, U17, V18, V19	-
	dinheiro_4 [34:28]	Display HEX4	PIN_U20, Y20, V20, U16, U15, Y15, P9	-
	dinheiro_5 [41:35]	Display HEX5	PIN_N9, M8, T14, P14, C1, C2, W19	-
	estado [4:0]	GPIO_0_D3, D5, D7, D9, D11	PIN_C16, K20, K22, M21, R22	StaticIO – LED – DIO 10, 11, 12, 13, 14
	rodada [3:0]	LEDR9, LEDR8, LEDR7, LEDR6	PIN_L1, L2, U1, U2	-

	contagem [3:0]	LEDR3, LEDR2, LEDR1, LEDR0	PIN_Y3, W2, AA1, AA2	-
	config_display [5:0]	GPIO_0_D25 [saldo = config_display 5], D27, D29, D31, D33, D35	PIN_K17, P18, R17, T20, T18, T15	-
	display_jogadas [11:0]	GPIO_0_D14, D22, D30, D26, D24, D16, D32, D28, D34, D20, D18, D12 a, b, c, d, e, f, g, dp, d1, d2, d3, d4 d4, a, f, d3, d2, b e, d, dp, c, g, d1 D12, D14, D16, D18, D20, D22, D24, D26, D28, D30, D32, D34	PIN_N20, L18, R16, K19, L19, M22, T19, R15, T17, P16, L22, R21	-
	display_rodadas [11:0]	d4, a, f, d3, d2, b e, d, dp, c, g, d1 0_D1, D3, D5, D7, D9, D11, D13, D15, D17, D19, D21, D23	PIN_C16, R22, P17, N19, T22, K20, M18, P19, L17, M21, K22, B16	-
	seletor_itens [9:0]	GPIO_1_D0 [seletor 9], D2, D4, D6, D8, D10, D12, D14, D16, D18 [seletor 0]	PIN_H16, H15, A13, C13, G18, H18, J19, H10, H14, J13	-
	indicador_itens [9:0]	GPIO_1_D1 [indicador_itens 9], D3, D5, D7, D9, D11, D13, D15, D17, D19 [indicador_itens 0]	PIN_A12, B12, B13, D13, G17, J18, G11, J11, A15, L8	-

Pinagem do ESP

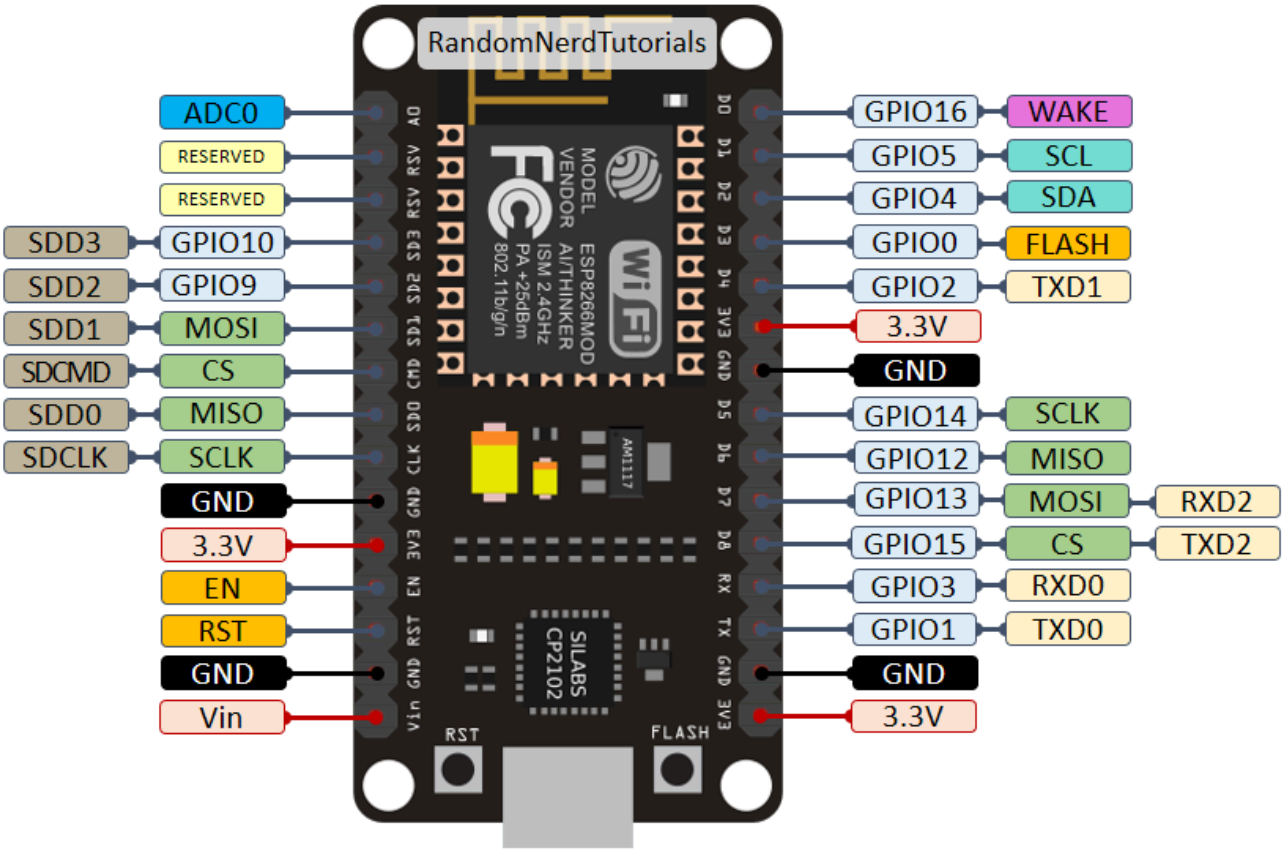


Imagem 19: Pinagem do módulo ESP32

Pino do Módulo RC522	Pino do ESP32 (GPIO)	Função
3.3V	**3.3V**	Alimentação (não use 5V para evitar danos)
GND	**GND**	Terra
SDA (SS)	**GPIO 5**	Seleção de Escravo (*Slave Select*)
SCK	**GPIO 18**	Clock Serial (*Serial Clock*)
MOSI	**GPIO 23**	Master Out Slave In
MISO	**GPIO 19**	Master In Slave Out
RST	**GPIO 21**	Reset
IRQ	*Não conectado*	Interrupção (opcional)

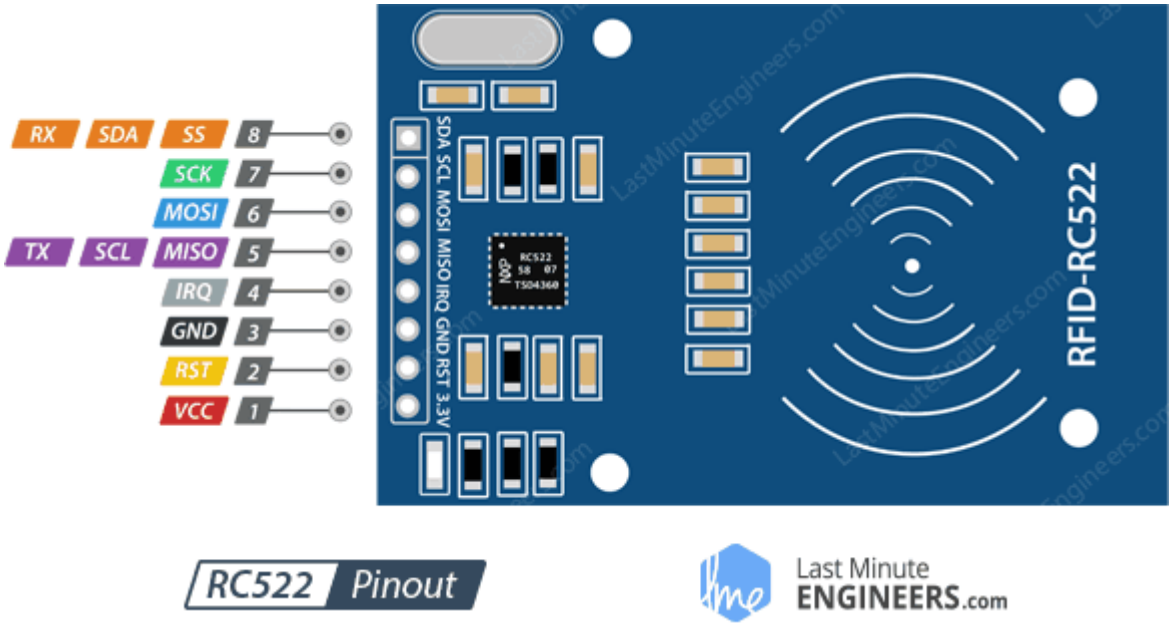


Imagem 20: Pinagem do leitor RFID-RC522

1.6.2 Estratégia de Montagem

Para essa montagem, foi necessário utilizar todos os elementos do jogo final. Para a montagem, utilizou-se a pinagem conforme o marcado na tabela, e ligou-se todos os fios dos botões, leds, etc. O Analog Discovery foi utilizado como gerador de clock e como input (substituindo a chave giratória). Além disso, todas as entradas e saídas foram retiradas da FPGA, exceto o estado, que foi deixado nos LEDRs. Foram colocados os leds para ligar no PERDEU, TERMINOU, ÚLTIMA JOGADA e ÚLTIMA RODADA, dessa forma, deixando o jogo já do jeito que será montado para a entrega final.

1.6.3 Estratégia de Depuração

A estratégia de depuração consiste em verificar o funcionamento geral do jogo, jogando-o por completo e verificando se os displays, leds, botões e conexões físicas estão todas funcionando corretamente, uma vez que a lógica do circuito já está validada.

1.6.4 Execução Prática do Cenário de Teste 1 – Teste físico do projeto Labdig 1 completo

Tabela – Descrição e Resultados Práticos do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Simulado OK?	Análise de Não Conformidades
resetar circuito	O circuito deve ser	OK	-

Entradas	Saídas Esperadas	Resultado Simulado OK?	Análise de Não Conformidades
	reiniciado		
iniciar ligado (=0) por 5 periodos de clock e esperar 10	O jogo deve ser iniciado	OK	-
executa ação **estudar** (jogada 1)	O saldo deve reduzir em 1000	OK	-
executa ação **trabalhar** (jogada 2)	O saldo deve aumentar em 1100	OK	-
altera temporariamente a configuração do display	Altera para mudar a visualização	OK	-
executa ação **estudar** (jogada 3)	O saldo deve reduzir em 1000	OK	-
executa ação **trabalhar** (jogada 4)	O saldo deve aumentar em 2100	OK	-
seleciona item “shopping” e realiza **compra**	O sistema deve aguardar a escolha do item	OK	A compra não foi executada, sistema não registrou.
seleciona “carro2” e realiza **compra**	O sistema deve registrar que o usuário possui o item, e descontar o valor do seu saldo	OK	A escolha do item não foi executada, sistema não registrou.
tenta comprar “carro2” novamente (não deve permitir)	O sistema não deve permitir comprar duas vezes	OK	-
seleciona “shopping” e realiza **venda**	O sistema deve ressarcir o valor do item comprado, e identificar que o jogador não possui mais o item	OK	-
executa ação **trabalhar** (jogada final)	O saldo deve aumentar em 2100	OK	-

1.7 Conclusão da Semana

O desenvolvimento realizado nesta primeira semana cumpriu integralmente o

objetivo de reconhecimento, auditoria técnica e nivelamento do projeto anterior. Em vez de avançar com implementações prematuras de código, a equipe concentrou os esforços na análise crítica da base herdada, mapeando gargalos na máquina de estados original e reavaliando a complexidade do sistema.

A partir desse diagnóstico, decidiu-se por uma simplificação estratégica do jogo de tabuleiro: a mecânica foi reestruturada para tornar as rodadas mais ágeis e acessíveis ao público-alvo, priorizando o fluxo de decisões essenciais de compra, economia e investimento. O foco tecnológico foi então redirecionado para a integração direta com o módulo leitor RFID (RC522), concretizando o ato de pagamento físico de forma literal e tangível.

Simultaneamente, definiu-se a arquitetura de comunicação serial (UART) com o microcontrolador ESP32 para publicação de eventos via MQTT, viabilizando a coleta de telemetria em tempo real e a análise de dados pedagógicos por meio do dashboard web em React. A caracterização elétrica de bancada do módulo RFID validou os níveis lógicos de 3,3 V e a pinagem do barramento SPI com a FPGA DE0-CV, estabelecendo uma base de hardware testada para que a síntese dos novos módulos e a transmissão de dados sejam iniciadas na Semana 2 com baixo risco de retrabalho.

2 SEMANA 2 DE DESENVOLVIMENTO DO PROJETO

Descrever sucintamente os objetivos da semana, referenciando requisitos e cronograma da especificação.

Caso a especificação tenha sido revisada, justificar sucintamente as mudanças neste ponto.

2.1 Descrição Funcional do Projeto para a Semana

Descrever tecnicamente o funcionamento esperado para o projeto nesta semana.

Sugestão: Utilizar fluxograma e/ou pseudocódigo e descrevê-lo em até dois parágrafos.

2.2 Detalhamento do Projeto Lógico do Hardware Programável

Esta seção do documento tem a finalidade de descrever detalhes do projeto lógico de hardware programável desenvolvido na Semana <1,2,3,4>.

2.2.1 Projeto do Fluxo de Dados

Detalhar o projeto do fluxo de dados.

Sugestão: incluir uma **figura legível** com o panorama do fluxo de dados e descrevê-lo em até dois parágrafos. Se houver questionamento ou necessidade de destaque de algum componente, seguir a mesma abordagem: adicionar figura e descrevê-la em até dois parágrafos.

2.2.2 Projeto da Unidade de Controle

Detalhar o projeto da Unidade de Controle.

Sugestão: incluir uma **figura legível** com o diagrama de transição de estados da unidade de controle e descrevê-la em uma tabela com a seguinte estrutura.

Tabela 5 – Descrição da Unidade de Controle do Sistema

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados

2.2.3 Projeto do Sistema Digital Programável

Descrever a integração entre fluxo de dados e unidade de controle. Incluir a descrição de **sinais de depuração** úteis para o projeto.

Sugestão: incluir **figura legível** e descrevê-la em até dois parágrafos.

2.3 Plano de Testes do Hardware Programável e Simulações

Este capítulo tem o objetivo de documentar a estratégia dos testes do sistema desenvolvido e documentado na seção 3.2 e apresentar os resultados das respectivas simulações realizadas.

Sugestão: descrever cada cenário de teste em uma subseção seguindo o modelo fornecido.

2.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura.

Tabela 6 – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Simulado OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

a) Criar uma coluna para cada entrada relevante;

b) Criar uma coluna para cada saída relevante;

c) Colunas “Resultado Simulado OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com resultados das simulações. **Incluir figuras legíveis das simulações após a tabela para corroborar os pontos levantados é importante.**

<Criar novas seções semelhantes à seção 3.3.1 para outros cenários de teste>

2.4 Detalhamento do Projeto de Software

Nesta seção do documento, são descritos detalhes do projeto de software desenvolvido na Semana <1,2,3,4>. Indicar como o software se relaciona ao hardware previsto no projeto.

Caso o grupo não tenha desenvolvimento de software na semana, ou no projeto como um todo, a seção e suas subseções podem ser excluídas.

2.4.1 Descrição da Arquitetura de Software

Detalhar a arquitetura de software (componentes previstos no projeto da semana) e as funcionalidades dos componentes projetados. Utilizar diagramas de blocos e/ou diagramas de casos de uso e classes.

2.4.2 Detalhamento do Projeto de Software

Incluir descrições dos algoritmos implementados nos componentes (ex.: fluxogramas e/ou diagramas de sequência com descrições textuais). Indicar eventuais bibliotecas utilizadas.

2.5 Plano de Testes do Software

Este capítulo tem o objetivo de documentar a estratégia dos testes de software desenvolvido e documentado na seção 3.4 e apresentar os seus resultados.

Sugestão: descrever cada cenário de teste em uma subseção seguindo o modelo fornecido.

2.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura.

Tabela 6 – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

a) Criar uma coluna para cada entrada relevante;

b) Criar uma coluna para cada saída relevante;

c) Colunas “Resultado OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com resultados dos testes. **Quando aplicável, incluir figuras legíveis de resultados relevantes.**

<Criar novas seções semelhantes à seção 3.3.1 para outros cenários de teste>

2.6 Integração entre Hardware Programável e Software

Nesta seção do documento, são descritos detalhes do processo de integração entre software e hardware desenvolvido na Semana <1,2,3,4>. Tal integração inclui a junção dos elementos e os respectivos testes integrados.

Se essa integração não for prevista na semana, ou no projeto como um todo, a seção pode ser excluída. Caso ela exista, é interessante descrever o processo de combinação de hardware e software e os testes previstos para validar que ela foi realizada com sucesso.

2.7 Prototipação do Projeto

Este capítulo tem o objetivo de documentar a construção das partes físicas do

projeto físico do projeto desenvolvidas na Semana <1,2,3,4>. O grupo deve incluir, quando aplicável, esquemas elétricos, modelos de peças mecânicas e diagramas de montagem e fiação aplicáveis ao desenvolvimento previsto na semana.

Incluir um diagrama mostrando as conexões físicas da placa FPGA com quaisquer componentes externos (exemplos: LEDs externos, botões externos, sensores, atuadores, placas microprocessadas). Incluir conexões com VCC e GND e, quando aplicável, o esquema elétrico da interligação entre os demais componentes.

2.8 Implantação do Projeto

Esta seção tem o objetivo de documentar as atividades práticas de execução do projeto desenvolvido e documentado no ambiente do Laboratório de Projeto de Sistemas Digitais.

2.8.1 Pinagem da Placa FPGA

Descrever em um parágrafo e incluir obrigatoriamente uma **figura legível** do "*pin planner*" do Intel Quartus evidenciando a realização da pinagem já no planejamento. A inclusão de uma tabela de pinagem desenvolvida à mão é opcional.

2.8.2 Estratégia de Montagem

Descrever o passo-a-passo para realizar a montagem experimental. Por exemplo:

a) Como realizar a programação da placa FPGA;

b) Como realizar ligações de terminais e fios **entre o projeto e os equipamentos de teste da bancada** (incluir diagramas/esquemas);

c) Quais sinais / elementos físicos são monitorados;

d) Quais sinais de depuração são considerados e como são monitorados.

Incluir um diagrama mostrando as conexões físicas do projeto com os elementos da bancada (exemplos: AnalogDiscovery, osciloscópio, fonte de alimentação, gerador de sinais)

2.8.3 Estratégia de Depuração

Descrever que estratégia o grupo prevê para realizar a depuração do projeto. Por exemplo:

a) Quais sinais de depuração foram incluídos no projeto, descrevendo um a um e justificando sua relevância;

b) Como monitorar cada sinal de depuração com os recursos do laboratório (osciloscópio, multímetro, módulos específicos do AnalogDiscovery, etc.) e como interpretá-los;

c) Como configurar os recursos do laboratório para as tarefas de depuração;

d) Como realizar alterações no projeto lógico em caso de problema;

e) Como realizar alterações de montagem em caso de problema;

f) Como garantir que as modificações não alteram outras funcionalidades já confirmadas como plenamente operacionais.

2.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura. **Se alterar algo em relação ao cenário de testes simulado, documentar explicitamente.**

Tabela 7 – Descrição e Resultados Práticos do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Prático OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

a) Criar uma coluna para cada entrada relevante;

b) Criar uma coluna para cada saída relevante;

c) Colunas “Resultado Prático OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com os resultados práticos. Sempre que o grupo julgar necessário, **incluir eventuais fotos para corroborar os pontos levantados**.

IMPORTANTE: A depender da complexidade do cenário de teste, o grupo pode substituir ou complementar a tabela com fotos e/ou vídeos da demonstração. Nesse caso, deve-se garantir que o professor tenha acesso a *links* compartilhados e deve-se elaborar um texto sucinto (sugestão: até dois parágrafos) que justifique a relação entre imagens/vídeos e a adequação dos resultados experimentais. Vídeos devem ser devidamente narrados, e vídeos e imagens devem ter resolução e foco para que as informações de interesse sejam legíveis e identificáveis.

<Criar novas seções semelhantes à seção 3.8.4 para outros cenários de teste (de hardware programável, de software e/ou e de integração entre hardware e software).>

2.9 Conclusão da Semana

No planejamento, **redija um texto de até um parágrafo explicando se o desenvolvimento realizado antes da aula cumpre os objetivos da semana**. Caso **falhas de projeto** existam, **descreva-as sucintamente.**

No relato final, **atualize as conclusões explicando se o desenvolvimento realizado na aula cumpre os objetivos da experiência e indique os próximos passos, se houver (Semanas 1 até 3).**

3 SEMANA 3 DE DESENVOLVIMENTO DO PROJETO

Descrever sucintamente os objetivos da semana, referenciando requisitos e cronograma da especificação.

Caso a especificação tenha sido revisada, justificar sucintamente as mudanças neste ponto.

3.1 Descrição Funcional do Projeto para a Semana

Descrever tecnicamente o funcionamento esperado para o projeto nesta semana.

Sugestão: Utilizar fluxograma e/ou pseudocódigo e descrevê-lo em até dois parágrafos.

3.2 Detalhamento do Projeto Lógico do Hardware Programável

Esta seção do documento tem a finalidade de descrever detalhes do projeto lógico de hardware programável desenvolvido na Semana <1,2,3,4>.

3.2.1 Projeto do Fluxo de Dados

Detalhar o projeto do fluxo de dados.

Sugestão: incluir uma **figura legível** com o panorama do fluxo de dados e descrevê-lo em até dois parágrafos. Se houver questionamento ou necessidade de destaque de algum componente, seguir a mesma abordagem: adicionar figura e descrevê-la em até dois parágrafos.

3.2.2 Projeto da Unidade de Controle

Detalhar o projeto da Unidade de Controle.

Sugestão: incluir uma **figura legível** com o diagrama de transição de estados da unidade de controle e descrevê-la em uma tabela com a seguinte estrutura.

Tabela 5 – Descrição da Unidade de Controle do Sistema

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados

3.2.3 Projeto do Sistema Digital Programável

Descrever a integração entre fluxo de dados e unidade de controle. Incluir a descrição de **sinais de depuração** úteis para o projeto.

Sugestão: incluir **figura legível** e descrevê-la em até dois parágrafos.

3.3 Plano de Testes do Hardware Programável e Simulações

Este capítulo tem o objetivo de documentar a estratégia dos testes do sistema desenvolvido e documentado na seção 3.2 e apresentar os resultados das respectivas simulações realizadas.

Sugestão: descrever cada cenário de teste em uma subseção seguindo o modelo fornecido.

3.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura.

Tabela 6 – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Simulado OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

d) Criar uma coluna para cada entrada relevante;

e) Criar uma coluna para cada saída relevante;

f) Colunas “Resultado Simulado OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com resultados das simulações. **Incluir figuras legíveis das simulações após a tabela para corroborar os pontos levantados é importante.**

<Criar novas seções semelhantes à seção 3.3.1 para outros cenários de teste>

3.4 Detalhamento do Projeto de Software

Nesta seção do documento, são descritos detalhes do projeto de software desenvolvido na Semana <1,2,3,4>. Indicar como o software se relaciona ao hardware previsto no projeto.

Caso o grupo não tenha desenvolvimento de software na semana, ou no projeto como um todo, a seção e suas subseções podem ser excluídas.

3.4.1 Descrição da Arquitetura de Software

Detalhar a arquitetura de software (componentes previstos no projeto da semana) e as funcionalidades dos componentes projetados. Utilizar diagramas de blocos e/ou diagramas de casos de uso e classes.

3.4.2 Detalhamento do Projeto de Software

Incluir descrições dos algoritmos implementados nos componentes (ex.: fluxogramas e/ou diagramas de sequência com descrições textuais). Indicar eventuais bibliotecas utilizadas.

3.5 Plano de Testes do Software

Este capítulo tem o objetivo de documentar a estratégia dos testes de software desenvolvido e documentado na seção 3.4 e apresentar os seus resultados.

Sugestão: descrever cada cenário de teste em uma subseção seguindo o modelo fornecido.

3.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura.

Tabela 6 – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

d) Criar uma coluna para cada entrada relevante;

e) Criar uma coluna para cada saída relevante;

f) Colunas “Resultado OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com resultados dos testes. **Quando aplicável, incluir figuras legíveis de resultados relevantes.**

<Criar novas seções semelhantes à seção 3.3.1 para outros cenários de teste>

3.6 Integração entre Hardware Programável e Software

Nesta seção do documento, são descritos detalhes do processo de integração entre software e hardware desenvolvido na Semana <1,2,3,4>. Tal integração inclui a junção dos elementos e os respectivos testes integrados.

Se essa integração não for prevista na semana, ou no projeto como um todo, a seção pode ser excluída. Caso ela exista, é interessante descrever o processo de combinação de hardware e software e os testes previstos para validar que ela foi realizada com sucesso.

3.7 Prototipação do Projeto

Este capítulo tem o objetivo de documentar a construção das partes físicas do

projeto físico do projeto desenvolvidas na Semana <1,2,3,4>. O grupo deve incluir, quando aplicável, esquemas elétricos, modelos de peças mecânicas e diagramas de montagem e fiação aplicáveis ao desenvolvimento previsto na semana.

Incluir um diagrama mostrando as conexões físicas da placa FPGA com quaisquer componentes externos (exemplos: LEDs externos, botões externos, sensores, atuadores, placas microprocessadas). Incluir conexões com VCC e GND e, quando aplicável, o esquema elétrico da interligação entre os demais componentes.

3.8 Implantação do Projeto

Esta seção tem o objetivo de documentar as atividades práticas de execução do projeto desenvolvido e documentado no ambiente do Laboratório de Projeto de Sistemas Digitais.

3.8.1 Pinagem da Placa FPGA

Descrever em um parágrafo e incluir obrigatoriamente uma **figura legível** do "*pin planner*" do Intel Quartus evidenciando a realização da pinagem já no planejamento. A inclusão de uma tabela de pinagem desenvolvida à mão é opcional.

3.8.2 Estratégia de Montagem

Descrever o passo-a-passo para realizar a montagem experimental. Por exemplo:

e) Como realizar a programação da placa FPGA;

f) Como realizar ligações de terminais e fios **entre o projeto e os equipamentos de teste da bancada** (incluir diagramas/esquemas);

g) Quais sinais / elementos físicos são monitorados;

h) Quais sinais de depuração são considerados e como são monitorados.

Incluir um diagrama mostrando as conexões físicas do projeto com os elementos da bancada (exemplos: AnalogDiscovery, osciloscópio, fonte de alimentação, gerador de sinais)

3.8.3 Estratégia de Depuração

Descrever que estratégia o grupo prevê para realizar a depuração do projeto. Por exemplo:

g) Quais sinais de depuração foram incluídos no projeto, descrevendo um a um e justificando sua relevância;

h) Como monitorar cada sinal de depuração com os recursos do laboratório (osciloscópio, multímetro, módulos específicos do AnalogDiscovery, etc.) e como interpretá-los;

i) Como configurar os recursos do laboratório para as tarefas de depuração;

j) Como realizar alterações no projeto lógico em caso de problema;

k) Como realizar alterações de montagem em caso de problema;

l) Como garantir que as modificações não alteram outras funcionalidades já confirmadas como plenamente operacionais.

3.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura. **Se alterar algo em relação ao cenário de testes simulado, documentar explicitamente.**

Tabela 7 – Descrição e Resultados Práticos do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Prático OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

d) Criar uma coluna para cada entrada relevante;

e) Criar uma coluna para cada saída relevante;

f) Colunas “Resultado Prático OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com os resultados práticos. Sempre que o grupo julgar necessário, **incluir eventuais fotos para corroborar os pontos levantados**.

IMPORTANTE: A depender da complexidade do cenário de teste, o grupo pode substituir ou complementar a tabela com fotos e/ou vídeos da demonstração. Nesse caso, deve-se garantir que o professor tenha acesso a *links* compartilhados e deve-se elaborar um texto sucinto (sugestão: até dois parágrafos) que justifique a relação entre imagens/vídeos e a adequação dos resultados experimentais. Vídeos devem ser devidamente narrados, e vídeos e imagens devem ter resolução e foco para que as informações de interesse sejam legíveis e identificáveis.

<Criar novas seções semelhantes à seção 3.8.4 para outros cenários de teste (de hardware programável, de software e/ou e de integração entre hardware e software).>

3.9 Conclusão da Semana

No planejamento, **redija um texto de até um parágrafo explicando se o desenvolvimento realizado antes da aula cumpre os objetivos da semana**. Caso **falhas de projeto** existam, **descreva-as sucintamente.**

No relato final, **atualize as conclusões explicando se o desenvolvimento realizado na aula cumpre os objetivos da experiência e indique os próximos passos, se houver (Semanas 1 até 3).**

4 SEMANA 4 DE DESENVOLVIMENTO DO PROJETO

Descrever sucintamente os objetivos da semana, referenciando requisitos e cronograma da especificação.

Caso a especificação tenha sido revisada, justificar sucintamente as mudanças neste ponto.

4.1 Descrição Funcional do Projeto para a Semana

Descrever tecnicamente o funcionamento esperado para o projeto nesta semana.

Sugestão: Utilizar fluxograma e/ou pseudocódigo e descrevê-lo em até dois parágrafos.

4.2 Detalhamento do Projeto Lógico do Hardware Programável

Esta seção do documento tem a finalidade de descrever detalhes do projeto lógico de hardware programável desenvolvido na Semana <1,2,3,4>.

4.2.1 Projeto do Fluxo de Dados

Detalhar o projeto do fluxo de dados.

Sugestão: incluir uma **figura legível** com o panorama do fluxo de dados e descrevê-lo em até dois parágrafos. Se houver questionamento ou necessidade de destaque de algum componente, seguir a mesma abordagem: adicionar figura e descrevê-la em até dois parágrafos.

4.2.2 Projeto da Unidade de Controle

Detalhar o projeto da Unidade de Controle.

Sugestão: incluir uma **figura legível** com o diagrama de transição de estados da unidade de controle e descrevê-la em uma tabela com a seguinte estrutura.

Tabela 5 – Descrição da Unidade de Controle do Sistema

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados

Nome do Estado	Descrição do Estado	Próximo Estado	Condições e Justificativas para a Transição entre Estados

4.2.3 Projeto do Sistema Digital Programável

Descrever a integração entre fluxo de dados e unidade de controle. Incluir a descrição de **sinais de depuração** úteis para o projeto.

Sugestão: incluir **figura legível** e descrevê-la em até dois parágrafos.

4.3 Plano de Testes do Hardware Programável e Simulações

Este capítulo tem o objetivo de documentar a estratégia dos testes do sistema desenvolvido e documentado na seção 3.2 e apresentar os resultados das respectivas simulações realizadas.

Sugestão: descrever cada cenário de teste em uma subseção seguindo o modelo fornecido.

4.3.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura.

Tabela 6 – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Simulado OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

g) Criar uma coluna para cada entrada relevante;

h) Criar uma coluna para cada saída relevante;

i) Colunas “Resultado Simulado OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com resultados das simulações. **Incluir figuras legíveis das simulações após a tabela para corroborar os pontos levantados é importante.**

<Criar novas seções semelhantes à seção 3.3.1 para outros cenários de teste>

4.4 Detalhamento do Projeto de Software

Nesta seção do documento, são descritos detalhes do projeto de software desenvolvido na Semana <1,2,3,4>. Indicar como o software se relaciona ao hardware previsto no projeto.

Caso o grupo não tenha desenvolvimento de software na semana, ou no projeto como um todo, a seção e suas subseções podem ser excluídas.

4.4.1 Descrição da Arquitetura de Software

Detalhar a arquitetura de software (componentes previstos no projeto da semana) e as funcionalidades dos componentes projetados. Utilizar diagramas de blocos e/ou diagramas de casos de uso e classes.

4.4.2 Detalhamento do Projeto de Software

Incluir descrições dos algoritmos implementados nos componentes (ex.: fluxogramas e/ou diagramas de sequência com descrições textuais). Indicar eventuais bibliotecas utilizadas.

4.5 Plano de Testes do Software

Este capítulo tem o objetivo de documentar a estratégia dos testes de software desenvolvido e documentado na seção 3.4 e apresentar os seus resultados.

Sugestão: descrever cada cenário de teste em uma subseção seguindo o modelo fornecido.

4.5.1 Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura.

Tabela 6 – Descrição e Resultados Simulados do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

g) Criar uma coluna para cada entrada relevante;

h) Criar uma coluna para cada saída relevante;

i) Colunas “Resultado OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com resultados dos testes. **Quando aplicável, incluir figuras legíveis de resultados relevantes.**

<Criar novas seções semelhantes à seção 3.3.1 para outros cenários de teste>

4.6 Integração entre Hardware Programável e Software

Nesta seção do documento, são descritos detalhes do processo de integração entre software e hardware desenvolvido na Semana <1,2,3,4>. Tal integração inclui a junção dos elementos e os respectivos testes integrados.

Se essa integração não for prevista na semana, ou no projeto como um todo, a seção pode ser excluída. Caso ela exista, é interessante descrever o processo de combinação de hardware e software e os testes previstos para validar que ela foi realizada com sucesso.

4.7 Prototipação do Projeto

Este capítulo tem o objetivo de documentar a construção das partes físicas do

projeto físico do projeto desenvolvidas na Semana <1,2,3,4>. O grupo deve incluir, quando aplicável, esquemas elétricos, modelos de peças mecânicas e diagramas de montagem e fiação aplicáveis ao desenvolvimento previsto na semana.

Incluir um diagrama mostrando as conexões físicas da placa FPGA com quaisquer componentes externos (exemplos: LEDs externos, botões externos, sensores, atuadores, placas microprocessadas). Incluir conexões com VCC e GND e, quando aplicável, o esquema elétrico da interligação entre os demais componentes.

4.8 Implantação do Projeto

Esta seção tem o objetivo de documentar as atividades práticas de execução do projeto desenvolvido e documentado no ambiente do Laboratório de Projeto de Sistemas Digitais.

4.8.1 Pinagem da Placa FPGA

Descrever em um parágrafo e incluir obrigatoriamente uma **figura legível** do "*pin planner*" do Intel Quartus evidenciando a realização da pinagem já no planejamento. A inclusão de uma tabela de pinagem desenvolvida à mão é opcional.

4.8.2 Estratégia de Montagem

Descrever o passo-a-passo para realizar a montagem experimental. Por exemplo:

a) Como realizar a programação da placa FPGA;

b) Como realizar ligações de terminais e fios **entre o projeto e os equipamentos de teste da bancada** (incluir diagramas/esquemas);

c) Quais sinais / elementos físicos são monitorados;

d) Quais sinais de depuração são considerados e como são monitorados.

Incluir um diagrama mostrando as conexões físicas do projeto com os elementos da bancada (exemplos: AnalogDiscovery, osciloscópio, fonte de alimentação, gerador de sinais)

4.8.3 Estratégia de Depuração

Descrever que estratégia o grupo prevê para realizar a depuração do projeto. Por exemplo:

a) Quais sinais de depuração foram incluídos no projeto, descrevendo um a um e justificando sua relevância;

b) Como monitorar cada sinal de depuração com os recursos do laboratório (osciloscópio, multímetro, módulos específicos do AnalogDiscovery, etc.) e como interpretá-los;

c) Como configurar os recursos do laboratório para as tarefas de depuração;

d) Como realizar alterações no projeto lógico em caso de problema;

e) Como realizar alterações de montagem em caso de problema;

f) Como garantir que as modificações não alteram outras funcionalidades já confirmadas como plenamente operacionais.

4.8.4 Execução Prática do Cenário de Teste X – <Objetivo Sucinto do Cenário>

Descrever cenário em até dois parágrafos e detalhar as características técnicas em uma tabela com a seguinte estrutura. **Se alterar algo em relação ao cenário de testes simulado, documentar explicitamente.**

Tabela 7 – Descrição e Resultados Práticos do Cenário de Teste 1

Entradas	Saídas Esperadas	Resultado Prático OK?	Análise de Não Conformidades

Notas de adaptação da tabela:

g) Criar uma coluna para cada entrada relevante;

h) Criar uma coluna para cada saída relevante;

i) Colunas “Resultado Prático OK?” e “Análise de Não Conformidades” devem ser preenchidas correlacionando-as com os resultados práticos. Sempre que o grupo julgar necessário, **incluir eventuais fotos para corroborar os pontos levantados**.

IMPORTANTE: A depender da complexidade do cenário de teste, o grupo pode substituir ou complementar a tabela com fotos e/ou vídeos da demonstração. Nesse caso, deve-se garantir que o professor tenha acesso a *links* compartilhados e deve-se elaborar um texto sucinto (sugestão: até dois parágrafos) que justifique a relação entre imagens/vídeos e a adequação dos resultados experimentais. Vídeos devem ser devidamente narrados, e vídeos e imagens devem ter resolução e foco para que as informações de interesse sejam legíveis e identificáveis.

<Criar novas seções semelhantes à seção 3.8.4 para outros cenários de teste (de hardware programável, de software e/ou e de integração entre hardware e software).>

4.9 Conclusão da Semana

No planejamento, **redija um texto de até um parágrafo explicando se o desenvolvimento realizado antes da aula cumpre os objetivos da semana**. Caso **falhas de projeto** existam, **descreva-as sucintamente.**

No relato final, **atualize as conclusões explicando se o desenvolvimento realizado na aula cumpre os objetivos da experiência e indique os próximos passos, se houver (Semanas 1 até 3).**

5 CONCLUSÕES DO PROJETO

No documento final, **redija um texto de até dois parágrafos explicando se o projeto final cumpre os objetivos previsto**. Caso **falhas de projeto** existam, **descreva-as sucintamente.**

Faça um breve balanço do processo de desenvolvimento, desde a especificação até a Feira de Projetos, e indique os principais desafios enfrentados e como o grupo lidou com eles.

Por fim, descreva **“sugestões para trabalhos futuros”, relatando possíveis melhorias ou próximos passos que podem ser realizados para a evolução do trabalho.**

6 REFERÊNCIAS BIBLIOGRÁFICAS

[1] OBERG, R.; PROBASCO, L.; ERICSSON, M.; *Applying Requirements Management with Use Cases*. Rational Software Corporation. Technical Paper TP505 (Version 1.4), 2001.

[2] A partir daqui, acrescente outras referências do seu projeto (ex.: artigos que justificam a importância do projeto).